Puzzlearm NVIDIA Jetson Orin/ Raspberry Pi Edition.

Introduction

Puzzlearm

{Learn, Create, Innovate};

Puzzlebot: NVIDIA Jetson Edition

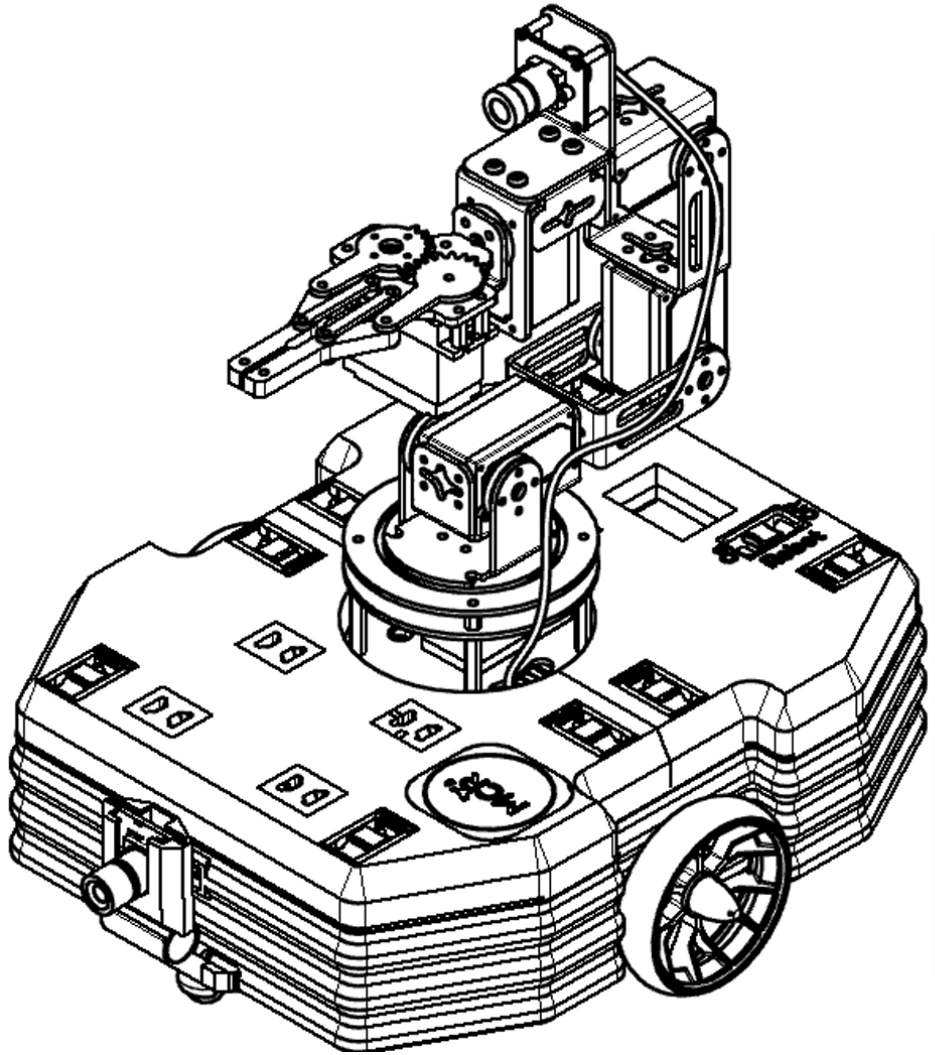
Introduction

The Puzzlearm is a compact educational robotics platform developed by Manchester Robotics. This edition contains a Yahboom Dofbot 6DOF manipulator and it is designed for hands-on learning in robotics, AI, and autonomous systems.

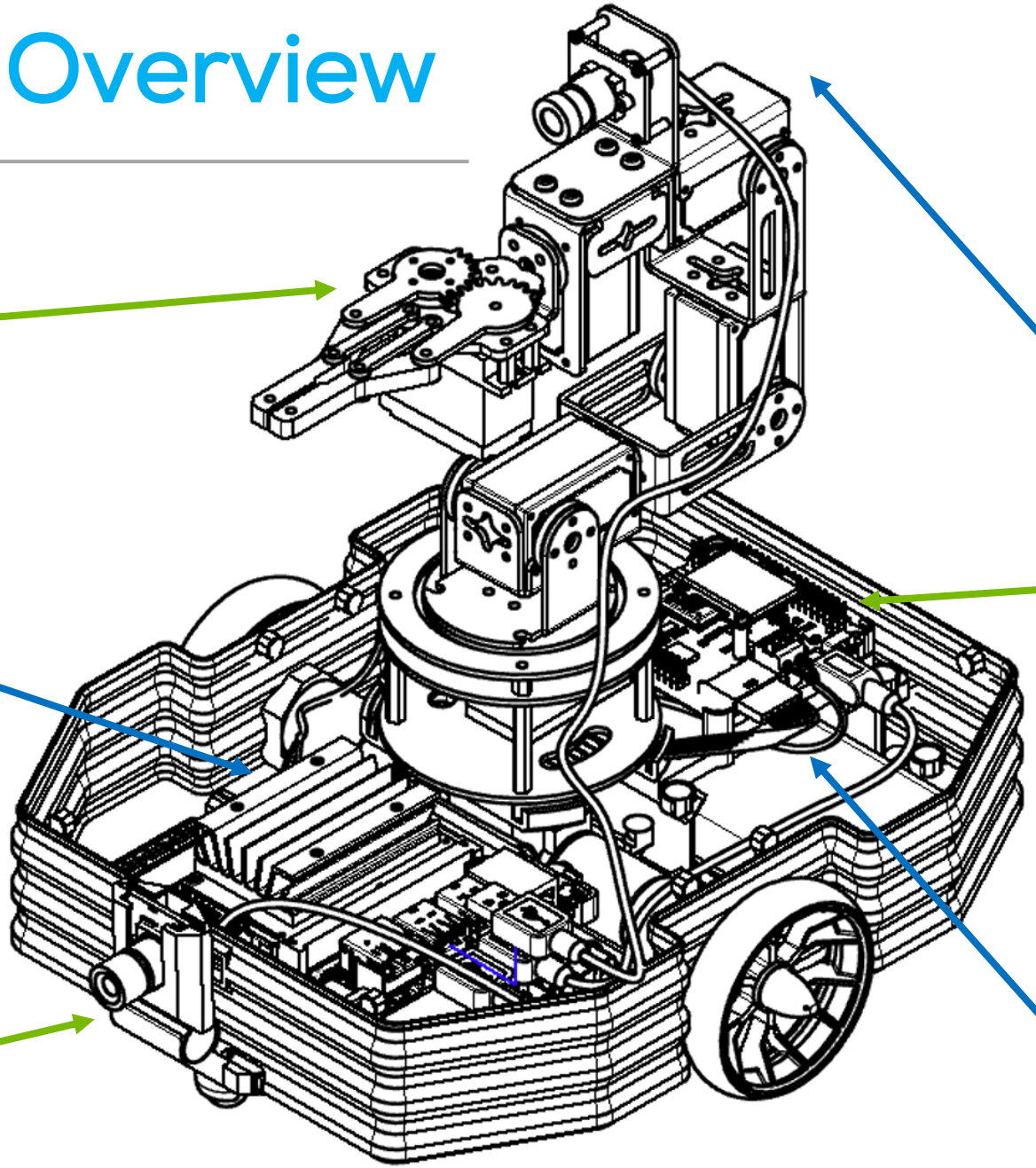
- Ideal for control, advanced robotics, computer vision and AI tasks.
- Features an integrated robotic arm to expand capabilities in manipulation and control.

Puzzlebot: NVIDIA Jetson ® Edition

- The following slides will guide the user through the basic usage of the Puzzlearm Jetson Orin Nano and Raspberry Pi 5 Edition
- The initial configuration will take place in two steps
  - The Hackeboard configuration.
  - The main computer configuration (Jetson Orin Nano or Raspberry Pi 5).



System Overview



Yahboom 6DoF Robotic Arm
For manipulation

NVIDIA Jetson Orin Nano / Raspberry Pi 5
For AI and control

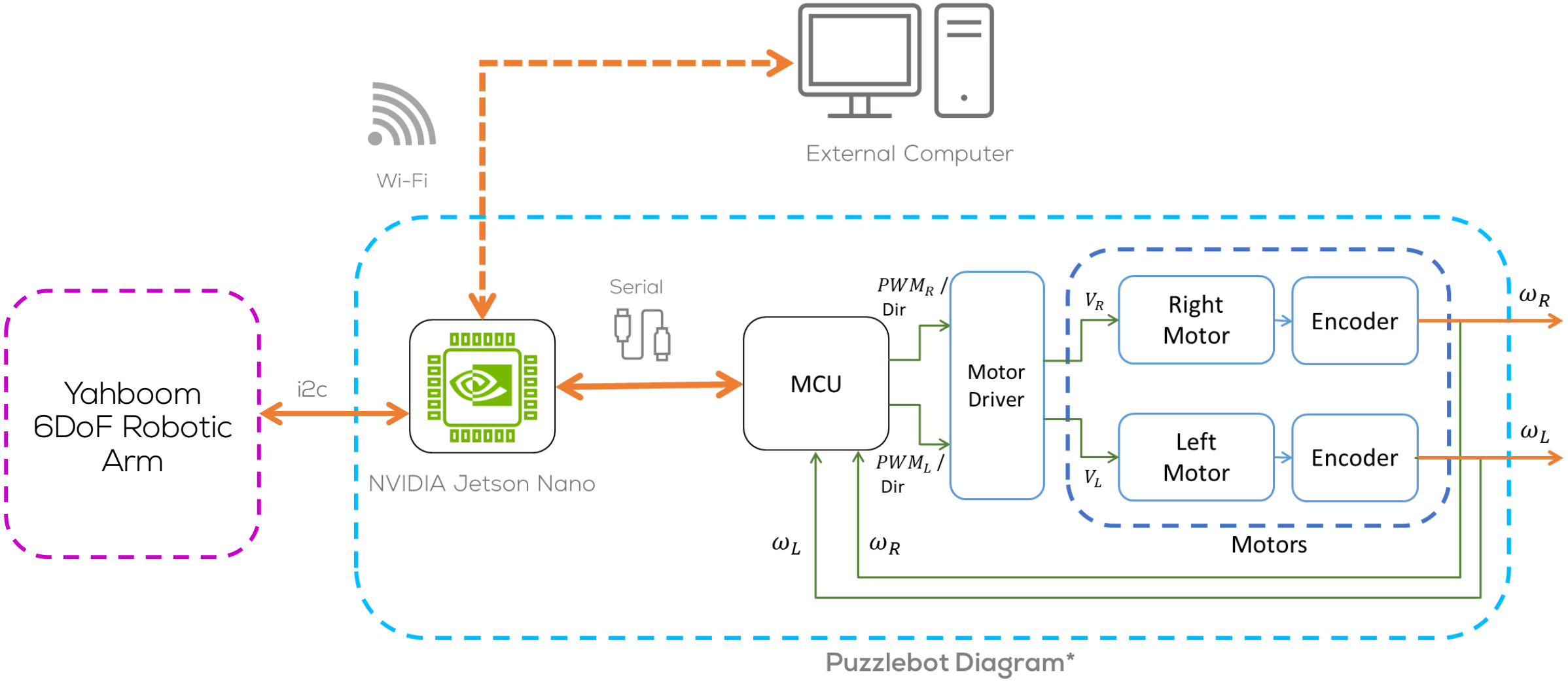
Main Camera
For computer vision

Arm Camera
For inspection

GPIO Arrays
Expansion is possible via the Hacker Board.

Hackerboard
For motor control using ROS2

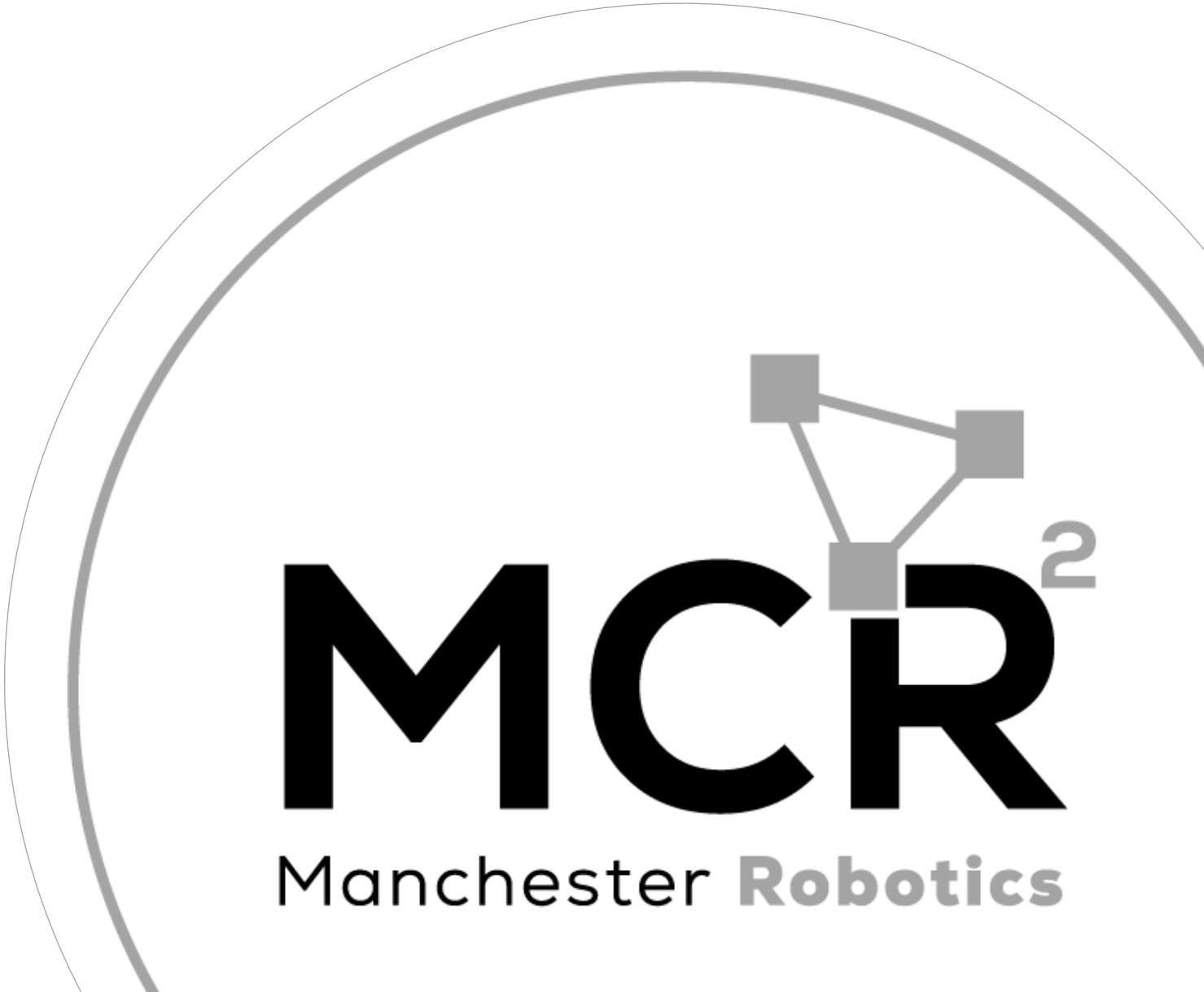
Puzzlebot Diagram



Hackerboard

Introduction

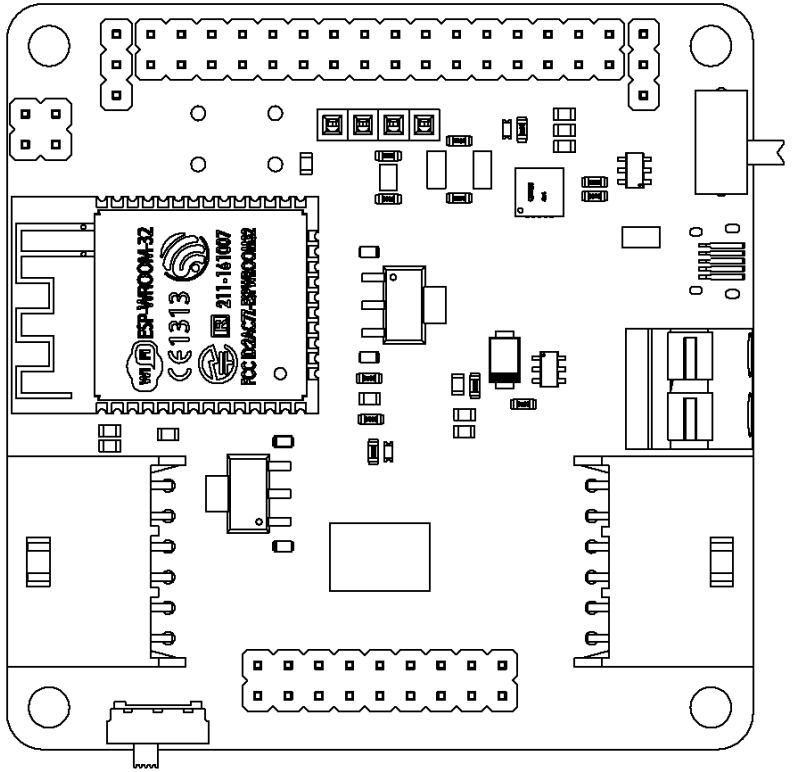
{Learn, Create, Innovate};



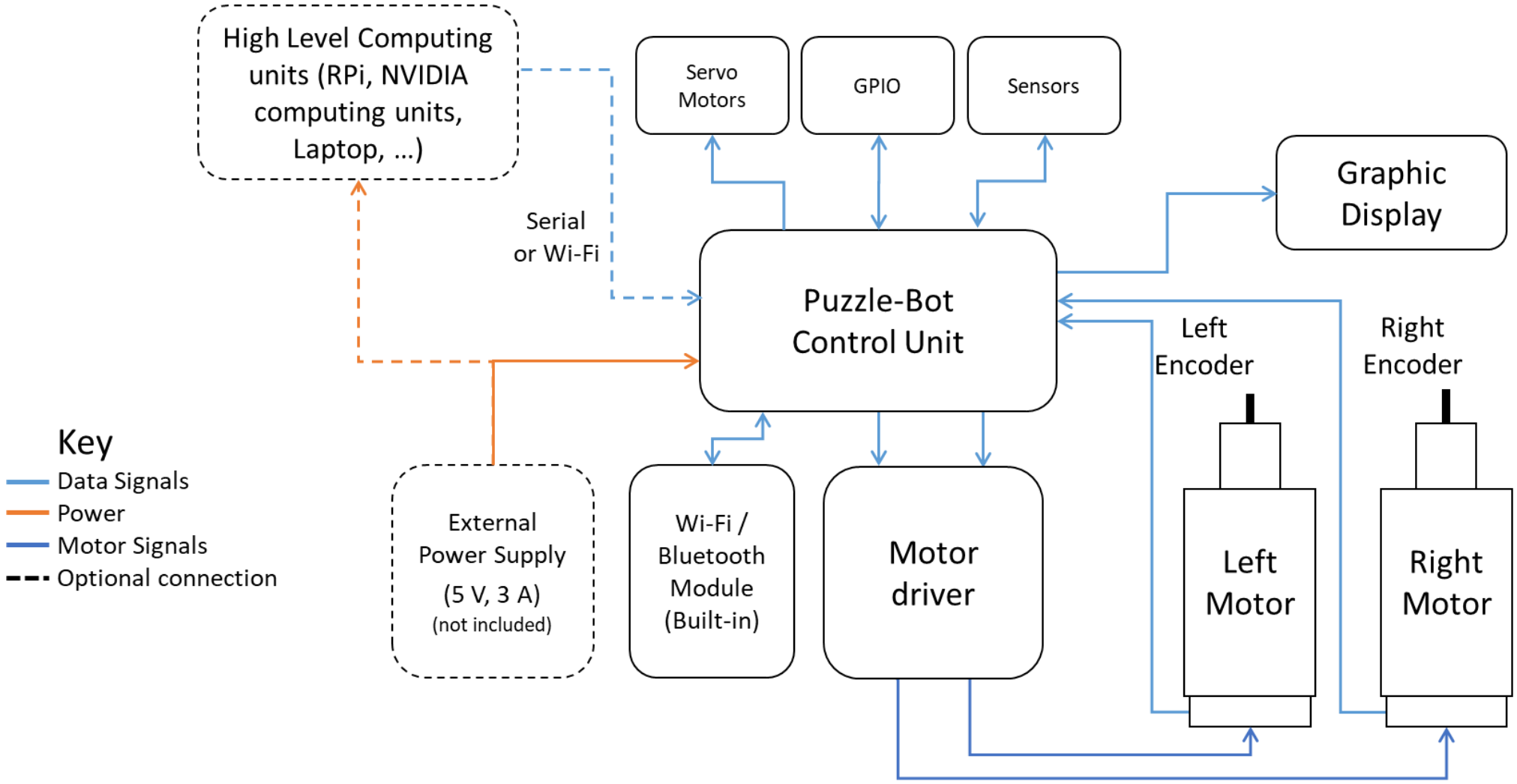
The Hackerboard

General characteristics

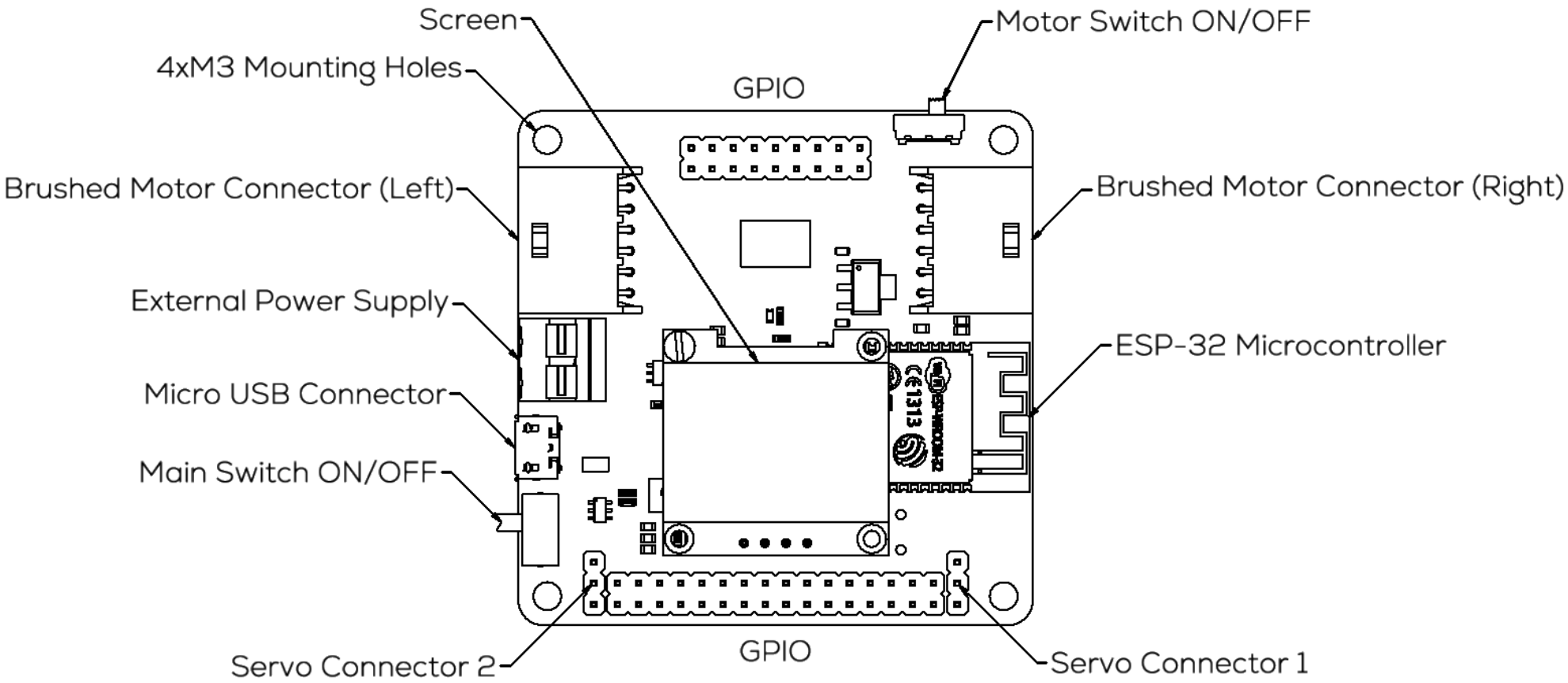
- ESP32-based Microcontroller
  - Xtensa dual-core 32-bit LX6 microprocessor
  - 520 KB of SRAM
  - Wi-Fi & Bluetooth
- DC-DC Converter
- Motor Driver
- 0.96” I2C LCD Display



System Architecture



The Hacker Board



For additional details, please refer to the pinout diagram.

Hackerboard

Control Modes

The Hackerboard has two different control modes depending on the user's requirements.

- The two programming configurations:
  - Standalone Configuration
  - External-Control Configuration

Control Mode: Standalone Configuration (Information purpose only)

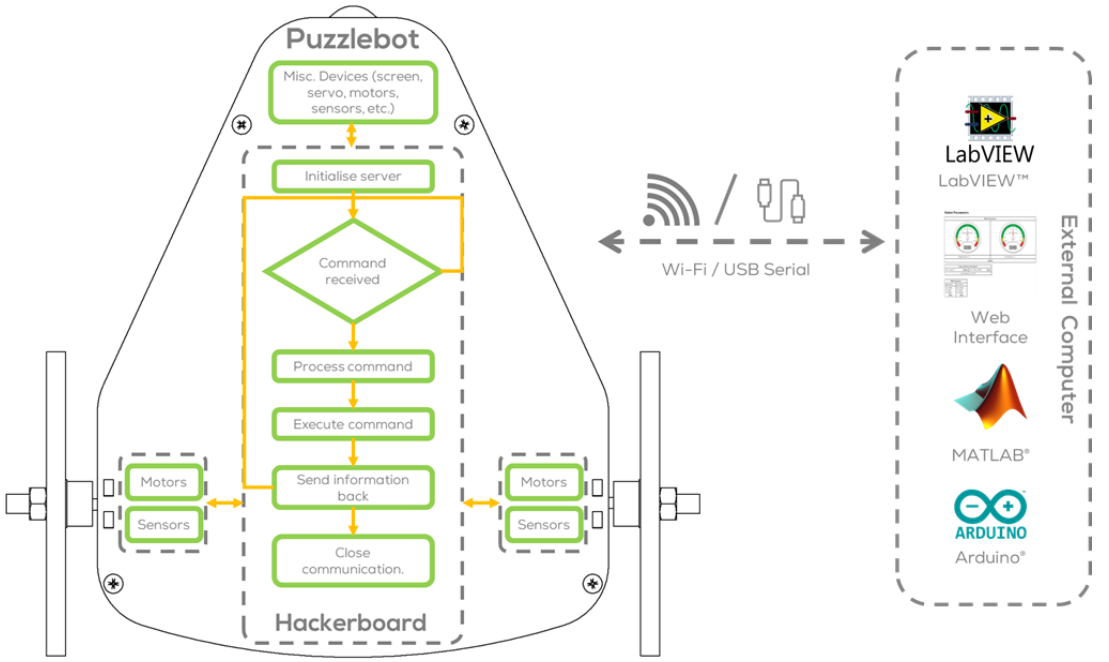
- The user directly programs the Hacker Board, using the Arduino IDE.
- Libraries for control and communication with computing units, sensors, and actuators are provided by MCR2.
- 3rd Party peripherals can be attached.
- This configuration will not be used for this Puzzlebot version.
- For additional examples and a more in-depth understanding, please consult the "Puzzlebot Hacker Edition" manual.

Hackerboard

Control Mode: External Control Configuration

The robot is controlled from an external computer via Wi-Fi or Serial Communication.

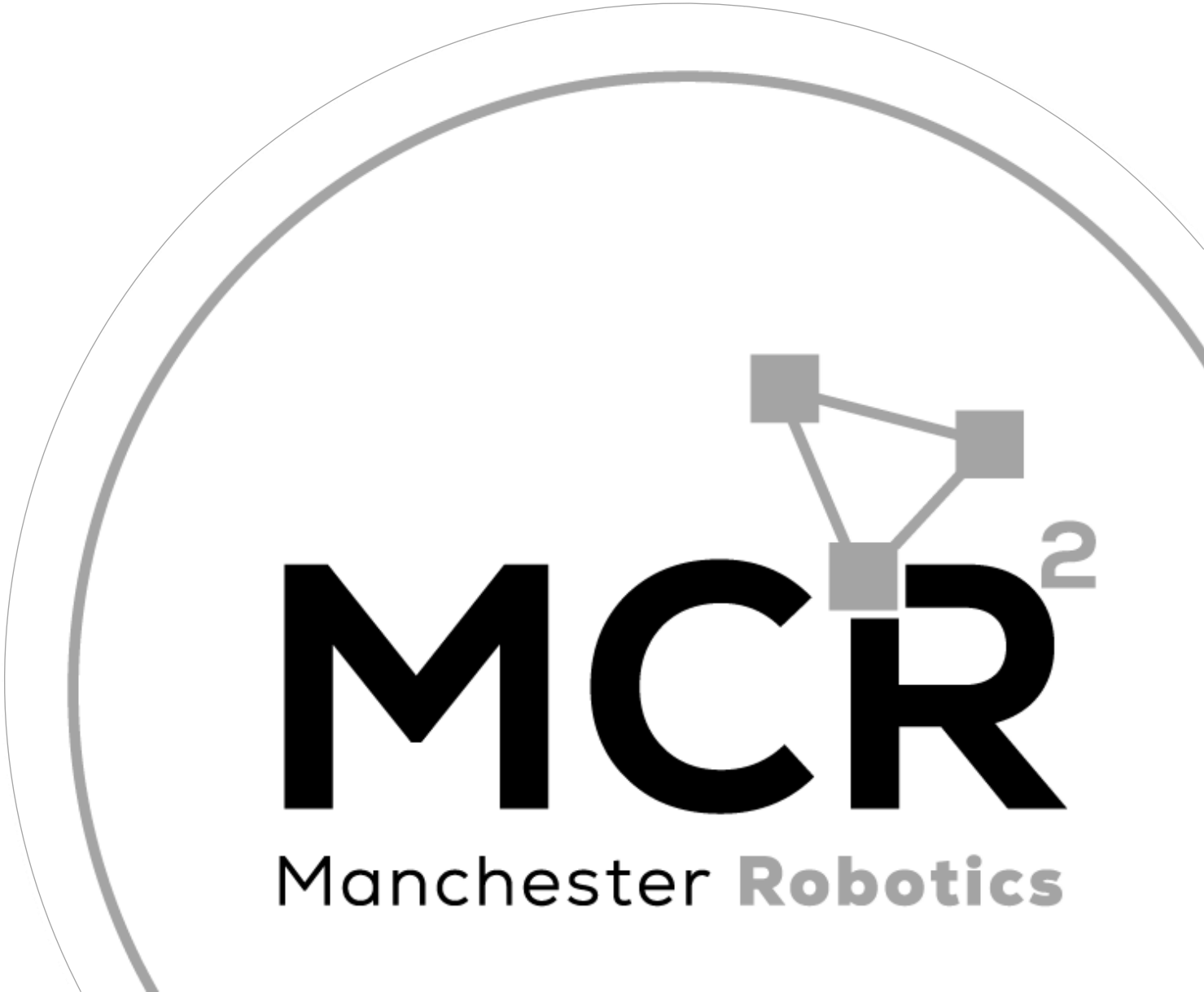
- The internal firmware and libraries for communicating with the robot's sensors and actuators are provided by MCR2.
- Basic web interface for configuring and testing provided.
- MCR2 provides MATLAB , ROS and LabVIEW libraries for communicating with the robot.
- This configuration will be used for this Puzzlebot version.
- For additional examples and a more in-depth understanding, on how to use MATLAB or LabVIEW communications and simulators, contact us.



Hackerboard

Flashing the Hackerboard

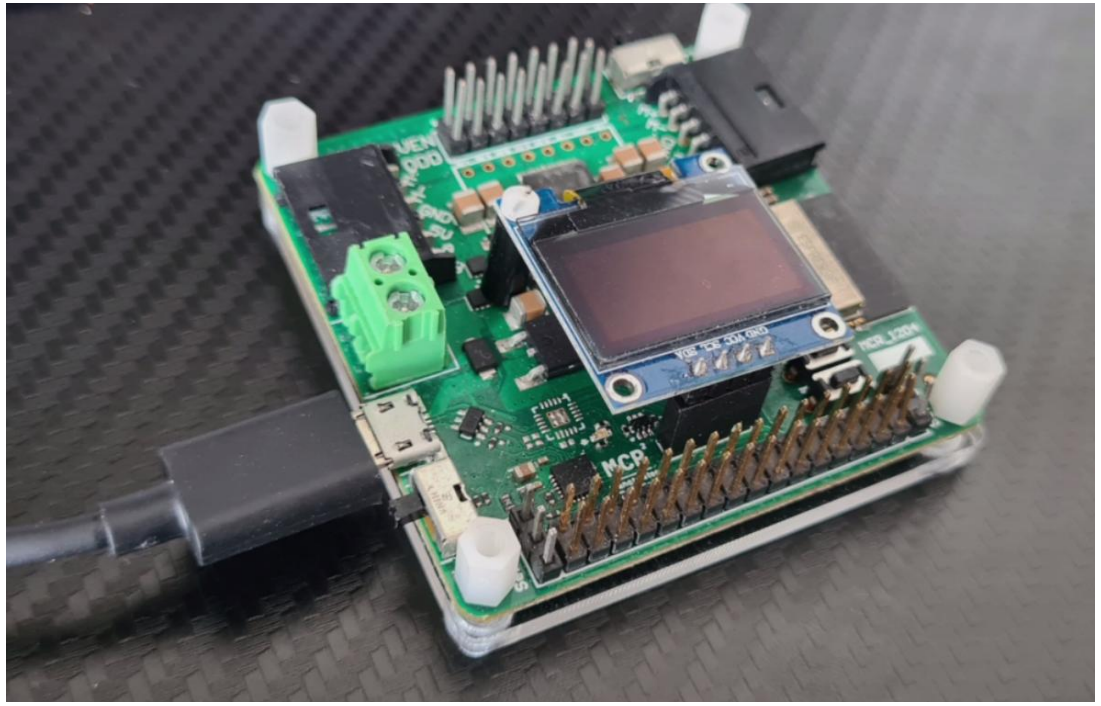
{Learn, Create, Innovate};



Hackerboard: Flashing the Binaries

Flashing Hackerboard

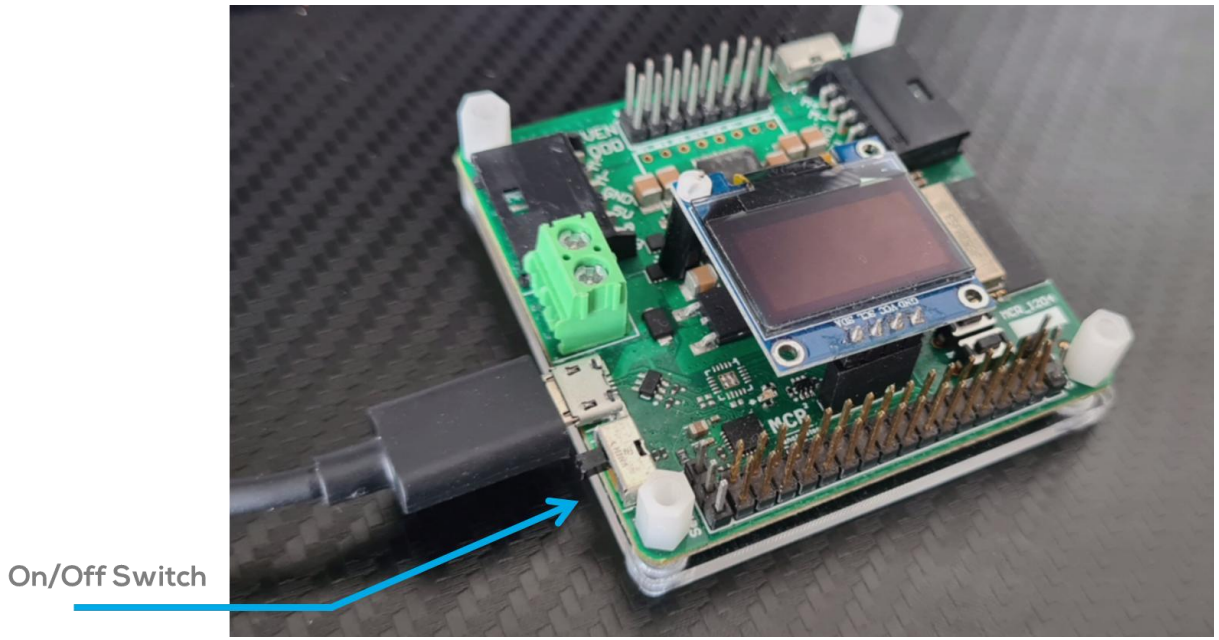
- MCR2 provides the firmware binaries for the External Computing Unit Control Mode.
- This section will guide the user on how to flash such binaries.
- All the robots come preprogrammed with such binaries unless the user modifies the program (On Board Configuration).
- The original binaries can be flashed anytime by following the steps in this section.



Hackerboard: Flashing the Binaries.

Steps

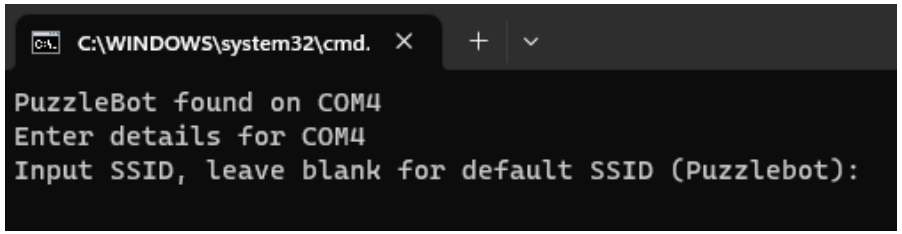
1. Attach the micro-USB cable to the Puzzle-Bot Control Module. *Note the image is just for explanation purposes, detachment of the Puzzle-Bot Control module from the robot is not necessary.*
2. Make sure the Hackerboard power switch is turned on.
3. Connect the USB to any free USB port in the computer
4. Download the firmware from the MCR2 GitHub and extract it.
5. Select the OS of the computer that will perform the flash.
6. Run the file *"FirmwareFlash"*
    - For Linux and MacOS users, it may be necessary to set execution permissions in properties.

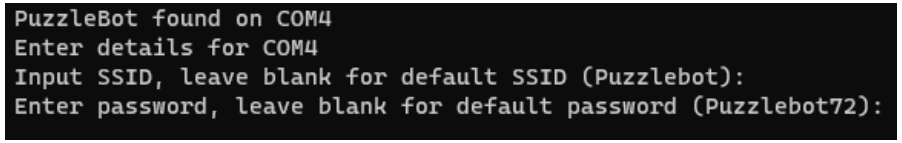


Hackerboard: Flashing the Binaries.

Steps

7. Select the SSID identification for the Hackerboard and press Enter (recommended to use a different one if several Puzzlebots are being used simultaneously).
8. Select the password for the Hackerboard's Wi-Fi Network and press Enter (recommended to use a different one if several Puzzlebots are being used simultaneously).
9. Wait until the program finish flashing.
10. The screen must turn on.



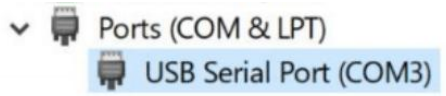




Troubleshoot

Troubleshoot (Drivers)

- Drivers are usually installed automatically by Windows and Ubuntu even for the Virtual Machines.
- How do I know if the drivers are properly installed (Windows)?
  - Plug the Puzzle-Bot into the USB port.
  - Go to Start › Device Manager
  - The Serial port should appear as shown in the following figure (The COM port may vary).



- If the computer cannot find the drivers, download the drivers from the following link

  https://ftdichip.com/drivers/vcp-drivers/

- Verify that the USB cable is a data cable and not only a power cable!

- Scroll down and download the executable setup as shown in the following figure



Operating System	Release Date	X86 (32-Bit)	X64 (64-Bit)	PPC	ARM	MIPSII	MIPSIV	SH4	Comments
Windows*	2021-07-15	2.12.36.4	2.12.36.4	-	-	-	-	-	WHQL Certified. Includes VCP and D2XX. Available as a setup executable Please read the Release Notes and Installation Guides.
Linux	-	-	1.5.0	-	-	-	-	-	All FTDI devices now supported in Ubuntu 11.10, kernel 3.0.0-19 Refer to TN-101 if you need a custom VCP VID/PID in Linux VCP drivers are integrated into the kernel.
Mac OS X 10.3 to 10.8	2012-08-10	2.2.18	2.2.18	2.2.18	-	-	-	-	Refer to TN-105 if you need a custom VCP VID/PID in MAC OS
Mac OS X 10.9 to 10.14	2019-12-24	-	2.4.4	-	-	-	-	-	This driver is signed by Apple

Before Installing the drivers!!

- Unplug the Puzzlebot from the computer.
- Unzip the drivers and run the setup (some computers must be restarted after the installation).
- Plug the Puzzlebot back into the computer.

Troubleshoot

Troubleshoot (Drivers)

- Some Hackerboard have a different USB-UART chip the CP210x.
- Drivers are usually installed automatically by Windows and Ubuntu even for the Virtual Machines.
- Verify if they are installed by following the steps in the previous slide.
- Verify that the USB cable is a data cable and not only a power cable!.
- If the computer cannot find the drivers, download the drivers from the following link

https://www.silabs.com/developers/usb-to-uart-bridge-vcp-drivers?tab=downloads

Before Installing the drivers!!

- Unplug the Puzzlebot from the computer.
- Unzip the drivers and run the setup (some computers are required to be restarted after the installation).
- Plug the Puzzlebot back into the computer.

A troubleshoot guide can be found here.

Troubleshoot

Troubleshoot (Drivers)

- My computer still not recognize the drivers even after the installation
- Plug the Puzzle-Bot into the USB port.
- Go to Start > Device Manager.
- Look for the USB Serial Converter as shown in the following picture.

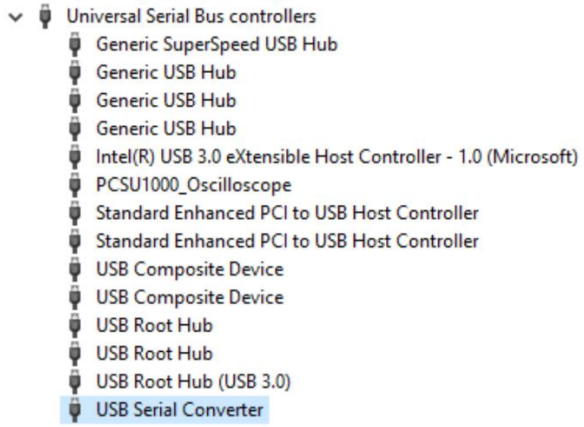


FIGURE: USB SERIAL CONVERTER

- Right Click to Properties > Advanced Tab.
- Make sure the Load VCP box is checked.
- Reconnect the Puzzle-Bot to the computer.

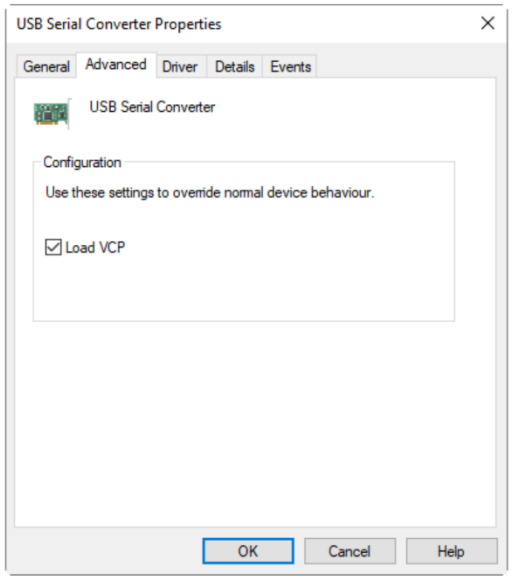


FIGURE: VCP PORT

Troubleshoot

Troubleshoot (Windows not flashing the hackerboard)

Try this approach when you are working on a Windows machine, and you still have at least one of the following problems.

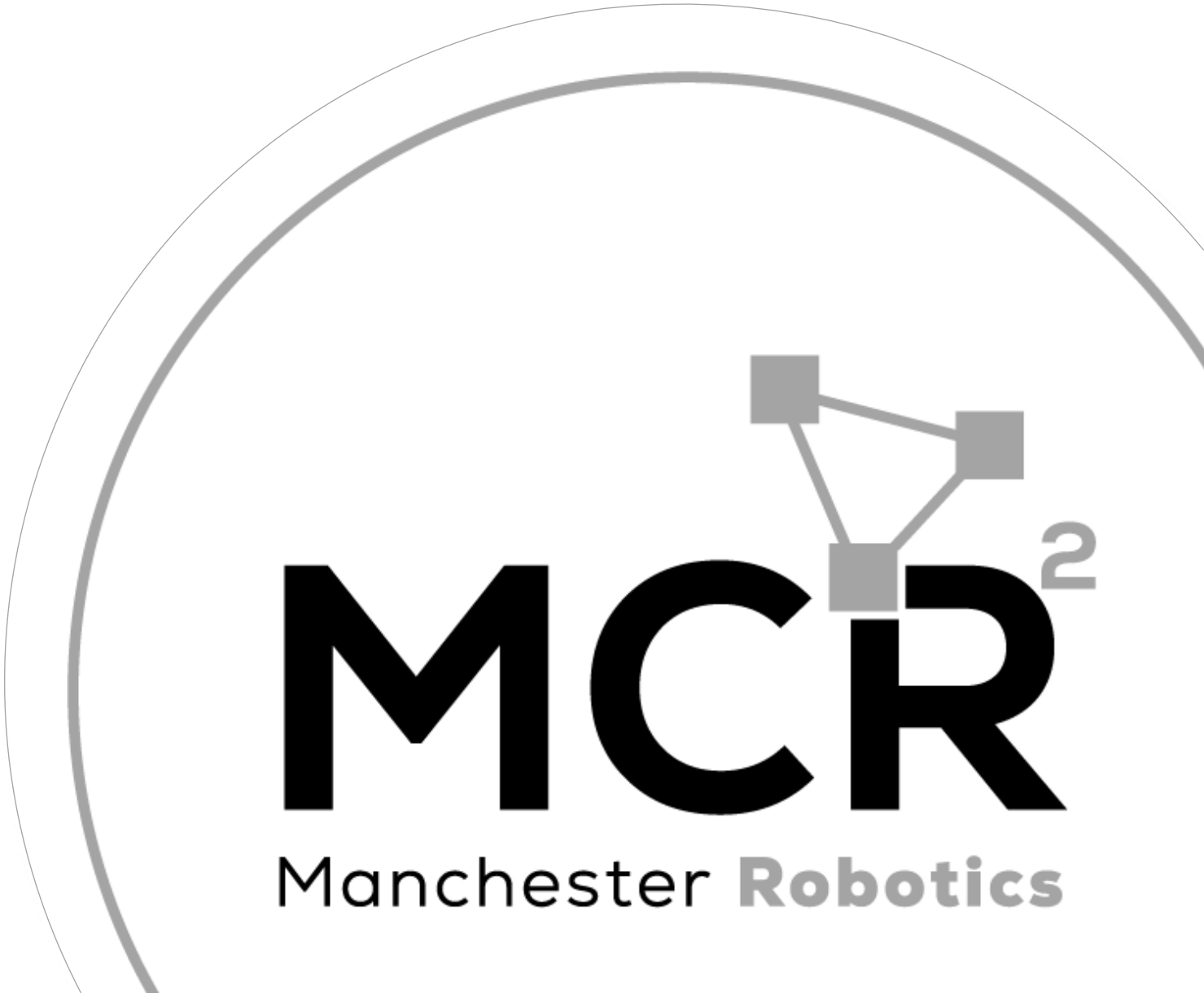
- My computer still not recognize the drivers even after the installation
- When I execute the FirmwareFlash file I can not find the COM device
- When I execute the FirmwareFlash, it says it is not an excecutable file.

1. Download MCR2 Virtual Machine from here.
2. Download the hackerboard binaries in the Virtual Machine.
3. Bridge the COM device of the hackerboard to the virtual machine.
4. Flash the hackerboard using the linux folder.

Hackerboard

Connections and configurations for the Jetson Orin Nano or the Raspberry Pi 5

{Learn, Create, Innovate};

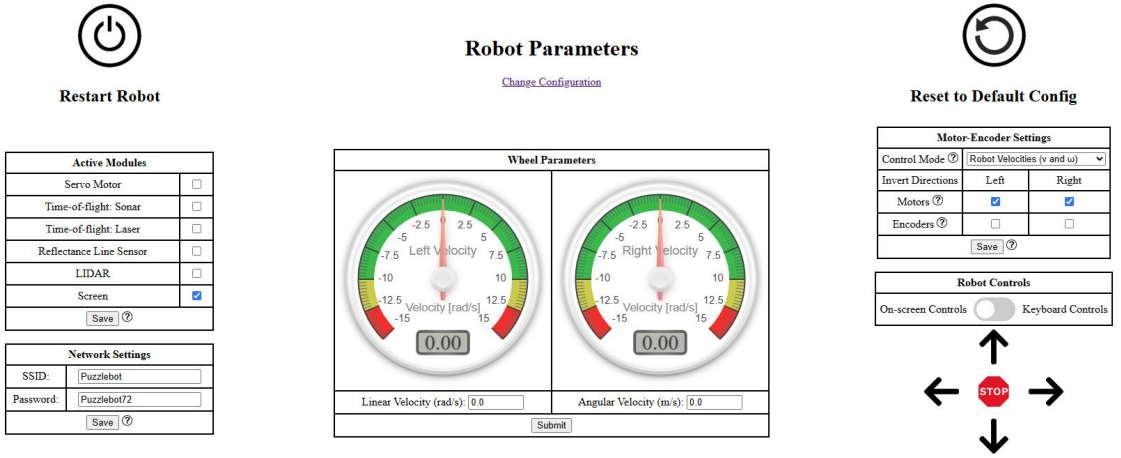


Connecting to the Hackerboard

Puzzlebot Web Interface

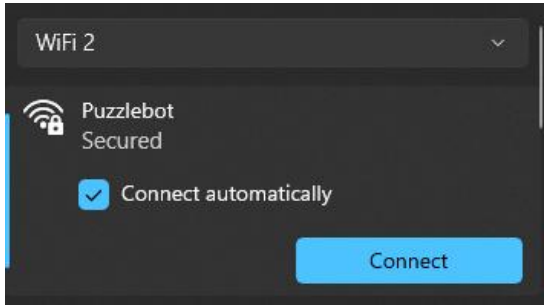
The Puzzlebot has a web interface allowing it to configure and test the different sensors and actuators equipped in the robot.

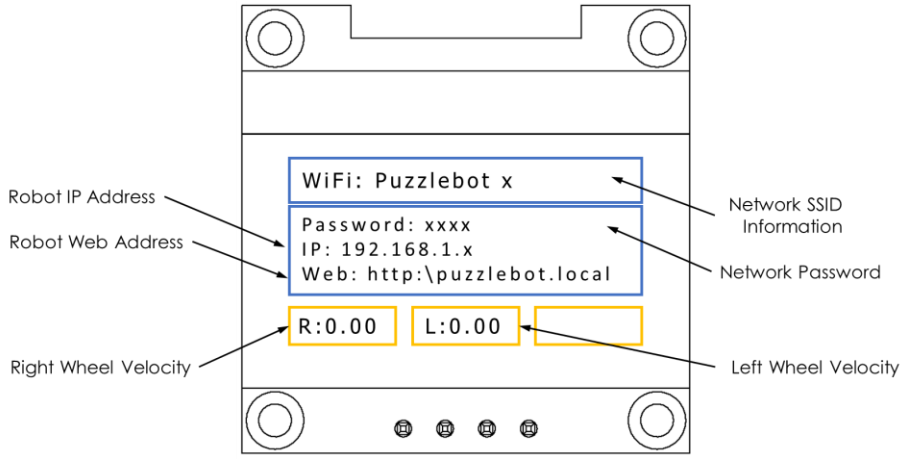
The website offers a visual interface that enables the user to configure and test various internal settings of the robot, including motor controllers, sensor activation, actuator activation (if applicable), and communication with external computing units.

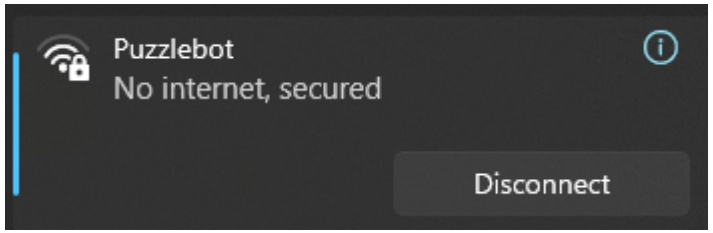


Connecting to the Hackerboard

1. Go to the Wi-Fi Network connections.
2. Connect to the Wi-Fi network created by the Puzzlebot by choosing the Puzzlebot Wi-Fi network of the robot you want (Puzzlebot x), then select Connect.
3. Type the network password that you selected when flashing (shown on the LCD Display), and then select Next.
4. Once Connected the network should say “No internet, secured” (Windows only).







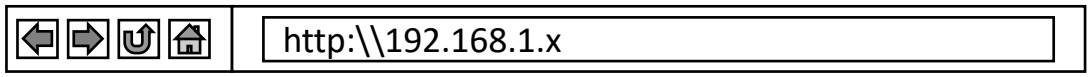
Connecting to the Hackerboard

Connecting to the Puzzlebot Web Interface

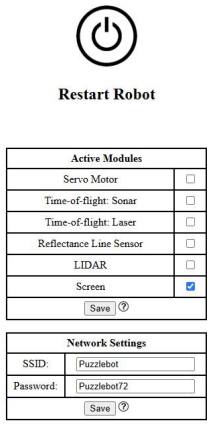
1. Open a web browser

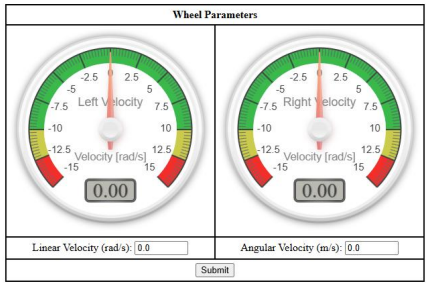


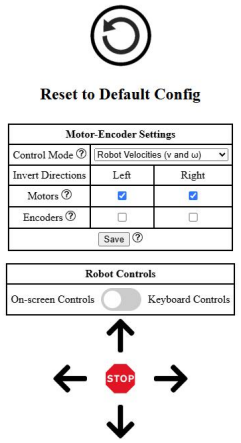
2. Type the following IP address on the search bar



3. The Puzzlebot Robot Parameters interface should load.



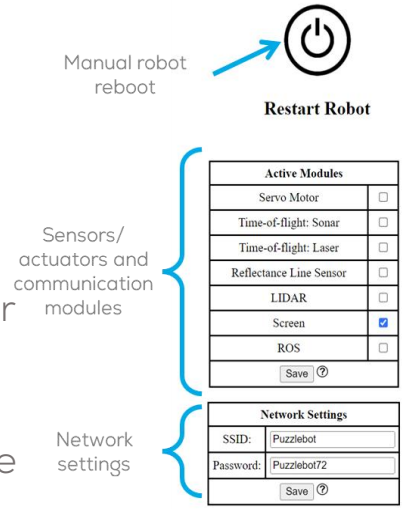


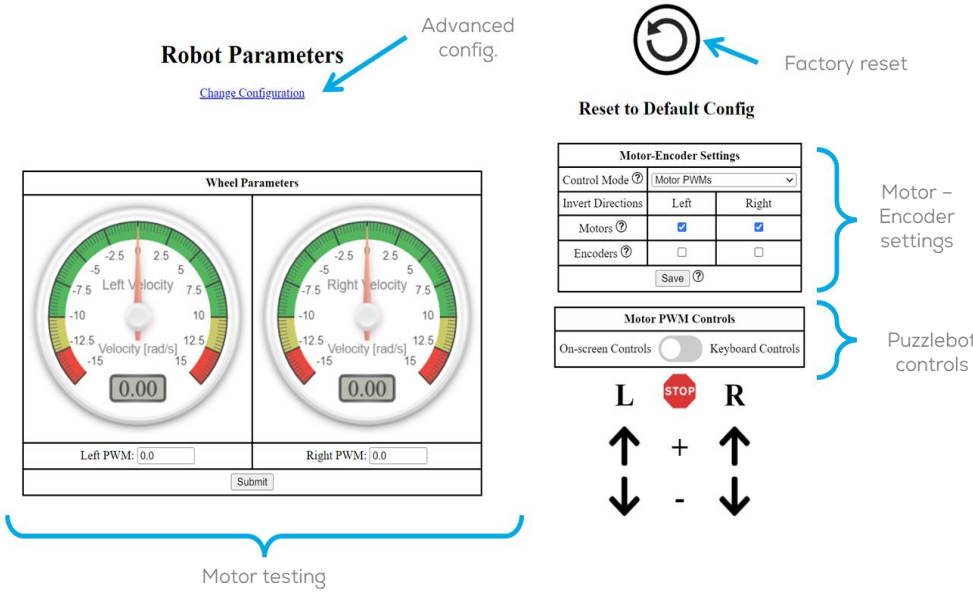


Configuring the Hackerboard

**This slide is only for informative purposes and is not required for basic Jetson connection.

- **Manual robot reboot:** Manually reboots the Hackerboard.
- **Factory Reset:** Reset all the configurations to factory configurations.
- **Network settings:** Configuration of the SSID and password of the Puzzlebot. Useful when multiple Puzzlebots are being used.
- **Puzzlebot controls:** Simple Puzzlebot controls for the user to move the robot forward, backwards or turning.
- **Sensor/actuators and comms. modules:** Activation of the sensors/actuators if included connected to the Hackerboard. The sensors and actuators in this section can connect to ROS. More information on the Puzzlebot Manual.

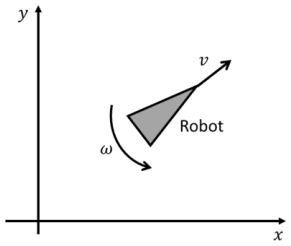




*Moving the cursor over the question marks (?) displays more information about each configuration parameter.

Configuring the Hackerboard

- **Advanced configuration:** Manually configure all aspects of the Puzzlebot. **Warning! adjusting the parameters of the Puzzlebot could result in significant malfunctions.**
- **Motor-Encoder Settings:** Motor and Encoder configurations.
  - Motor PWM: Values in the range of [-1, 1] (No control)
  - Wheel velocities (ωL and ωR): Wheel velocities (Inner PID control used)
  - Robot Velocities (v and ω): Linear and angular speed of the robot.



- **Motor testing:** Manually send values to the motors for testing. The values sent depend on the Control Mode.

- **Advanced configuration:** Manually configure all aspects of the Puzzlebot. **Warning! adjusting the parameters of the Puzzlebot could result in significant malfunctions.**
- **Motor-Encoder Settings:** Motor and Encoder configurations.
  - Motor PWM: Values in the range of [-1, 1] (No control)
  - Wheel velocities (ωL and ωR): Wheel velocities (Inner PID control used)
  - Robot Velocities (v and ω): Linear and angular speed of the robot.
- **Motor testing:** Manually send values to the motors for testing. The values sent depend on the Control Mode.

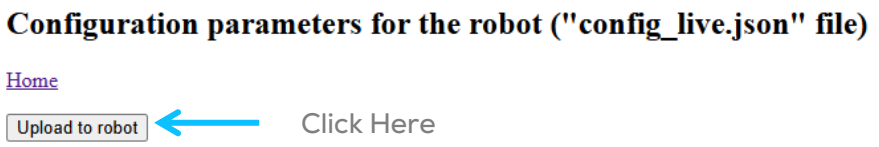
Connecting to the Hackerboard

Configuring ROS Serial communication

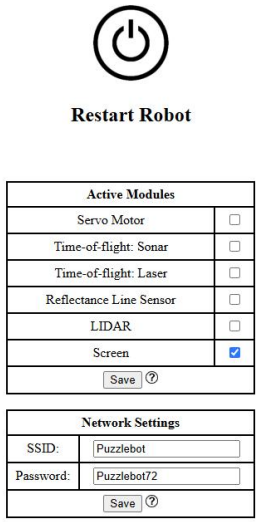
1. Click on the “Change Configuration” link on the top. This will take you to another website were all the internal configurations of the robot can be seen.

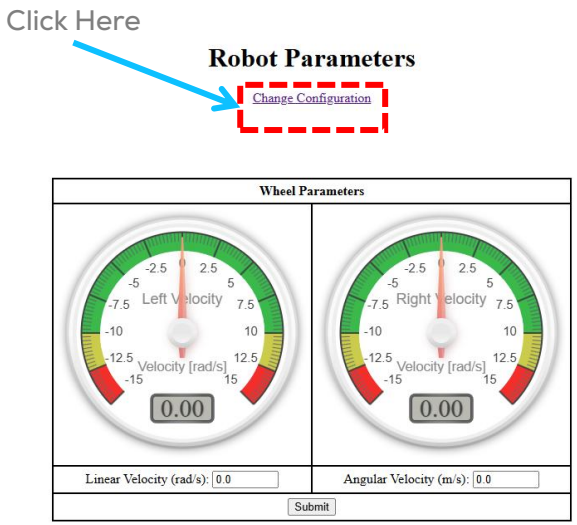
2. In the section “Ros” Change the value of “CommType” to “2”.

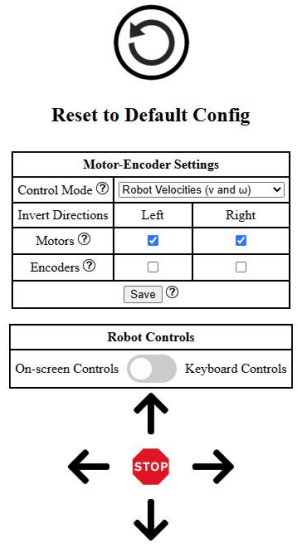
3. Upload to the robot by pressing the button “Upload to the robot” at the top of the webpage.



4. The Hackerboard will restart








```
"Ros": {
  "MasterIP": [
    192,
    168,
    1,
    2
  ],
  "CommType": 2,
  "MasterPort": 9999,
  "Dt": 0.005
},
```

Connecting to the Hackerboard

Configuring the Hackerboard

4. The ROS topics to be published by the Puzzlebot depend on the “Control Mode” Selected.
    - See table in next slide.
5. Having selected the Control Mode and activated the ROS Module, press Save and disconnect from the robot.

Motor-Encoder Settings		
Control Mode ⓘ	Robot Velocities (v and ω)	
Invert Directions	Left	Right
Motors ⓘ	☑	☑
Encoders ⓘ	☐	☐
Save ⓘ		

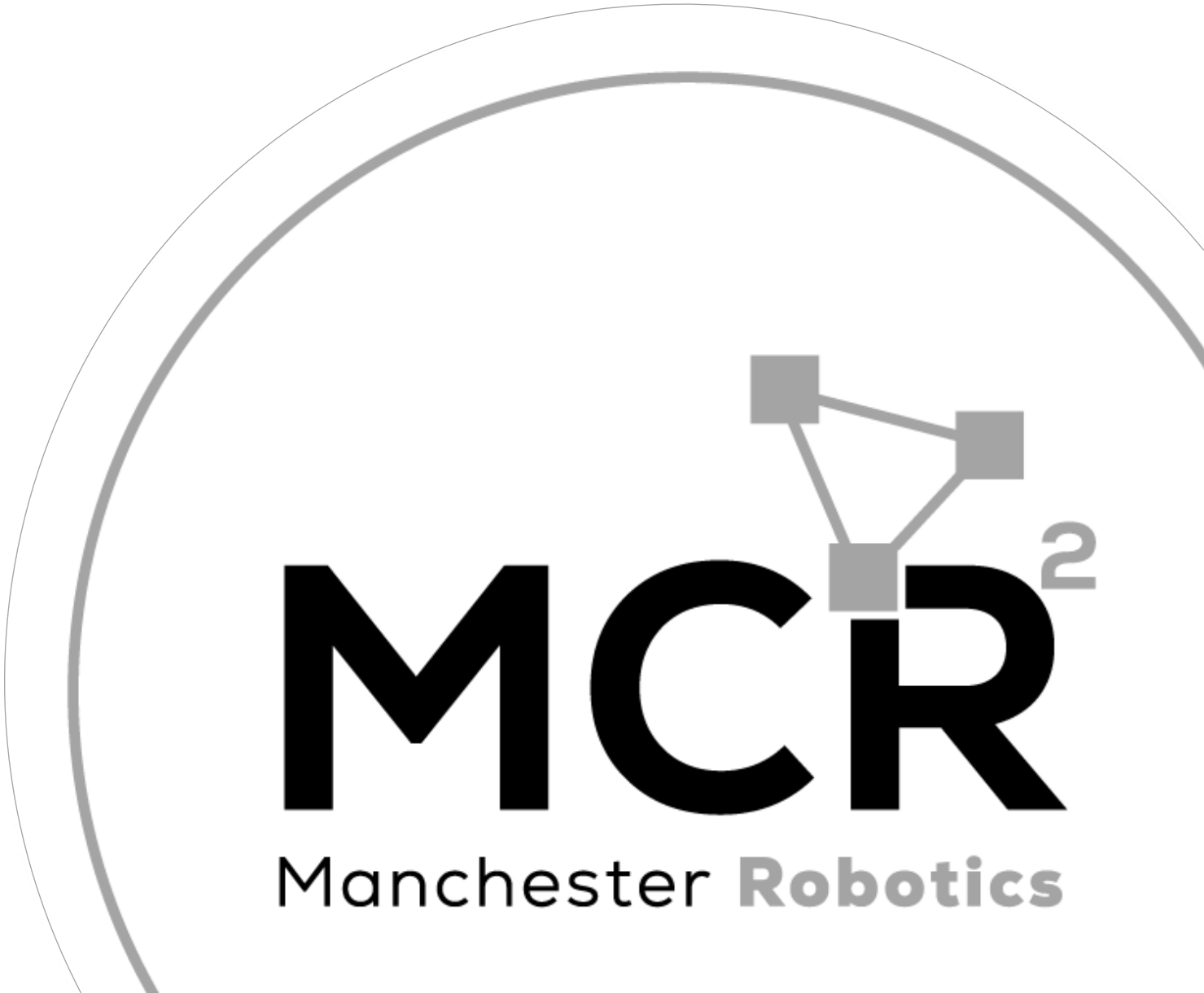
Connecting to the Hackerboard

Control Mode	Description	Topic	Type	Information
Motor PWMs	Wheel PWM voltage signal	/ControlL, /ControlR	std_msgs/Float32	data: PWM duty cycle for each motor [-1,1]
Wheel velocities (ωL and ωR)	Wheel angular velocities setpoint (PID control)	/VelocitySetR, /VelocitySetL	std_msgs/Float32	data: Control Set Point for wheel velocities
Robot Velocities (v and ω)	Linear and Angular Velocities (PID control)	/cmd_vel	geometry_msgs/Twist	linear x: Linear speed of the robot y: Not used z: Not Used angular: x: Not Used y: Not Used z: Angular Speed of the robot

Raspberry Pi 5

Configurations

{Learn, Create, Innovate};



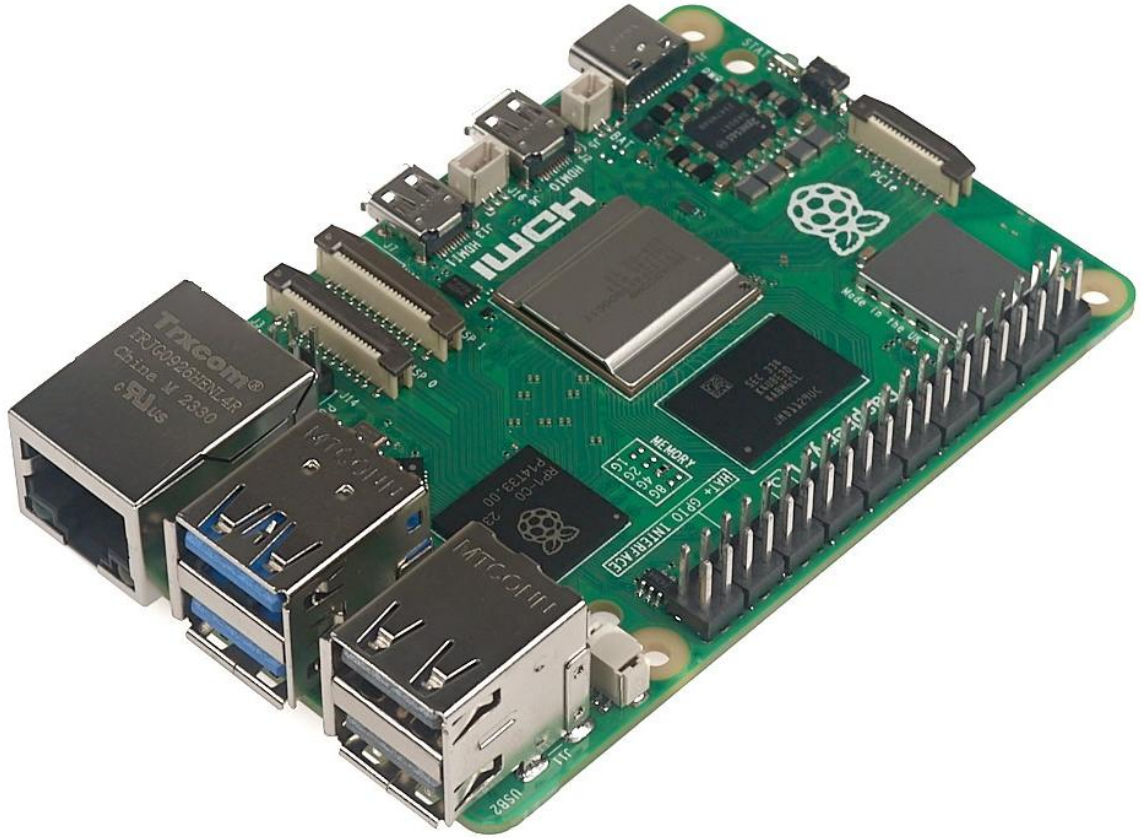
Raspberry Pi 5 16 GB

General characteristics

- **Processor**: 2.4 GHz quad-core 64-bit Arm Cortex-A76 CPU.
- **Graphics**: VideoCore VII GPU supporting OpenGL ES 3.1 and Vulkan 1.2.
- **RAM Options**: Available in 2GB, 4GB, 8GB, or 16GB LPDDR4X-4267 SDRAM configurations.

- **Display Output**: Dual 4K@60Hz HDMI outputs with HDR support.
- **USB Ports**: 2 × USB 3.0 ports (supporting simultaneous 5Gbps operation) and 2 × USB 2.0 ports.
- **Peripherals**: 1 × PCIe 2.0 x1 interface for high-speed accessories (requires separate M.2 HAT for NVMe SSDs).
- **Camera/Display**: 2 × 4-lane MIPI camera/display transceivers.

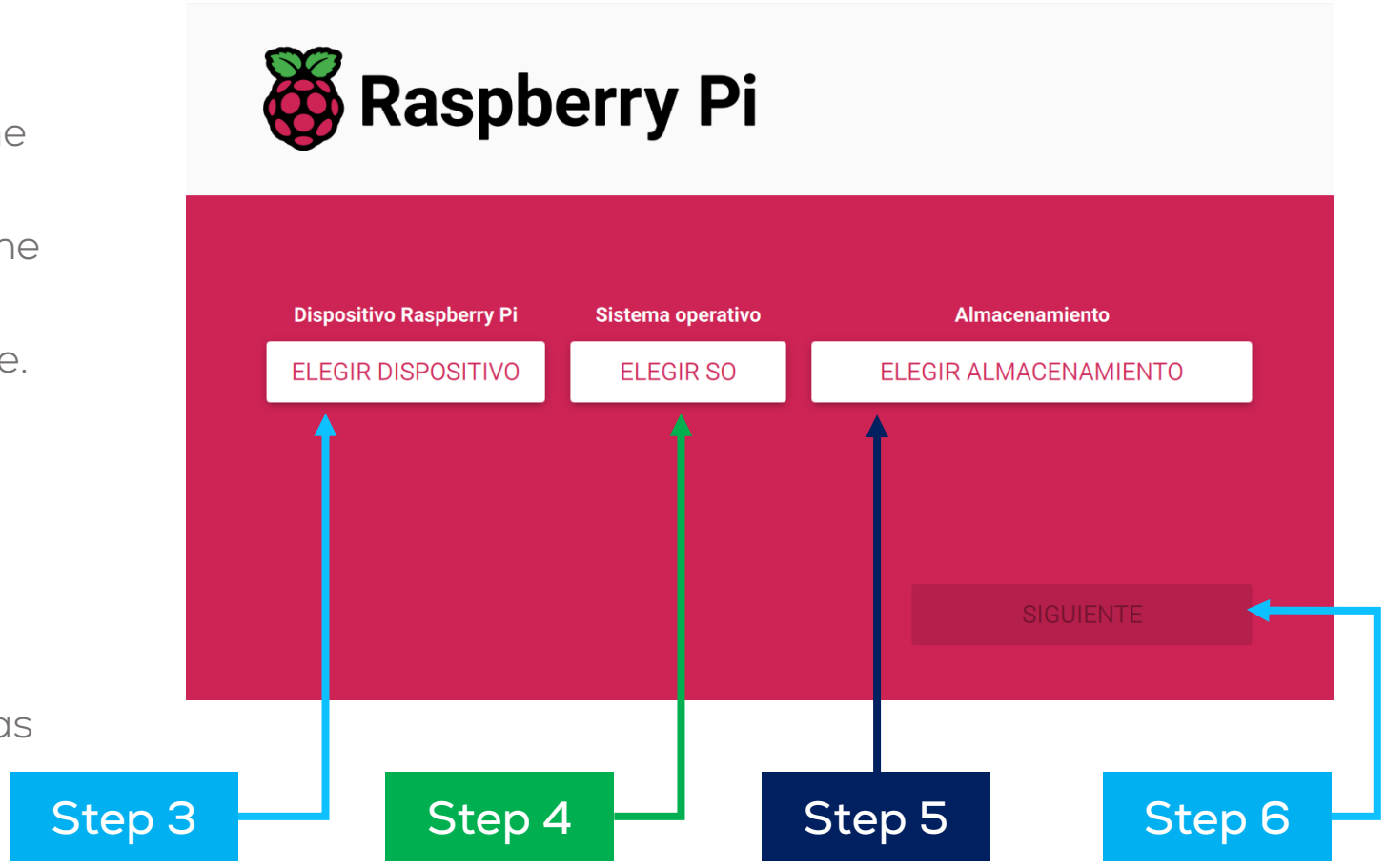
- **Wi-Fi**: Dual-band 802.11ac Wi-Fi (2.4 GHz and 5.0 GHz).
- **Bluetooth**: Bluetooth 5.0 and Bluetooth Low Energy (BLE).
- **Ethernet**: Gigabit Ethernet with PoE+ support (requires separate PoE+ HAT).



RPi5 OS Installation

Ubuntu 24.04 install instructions

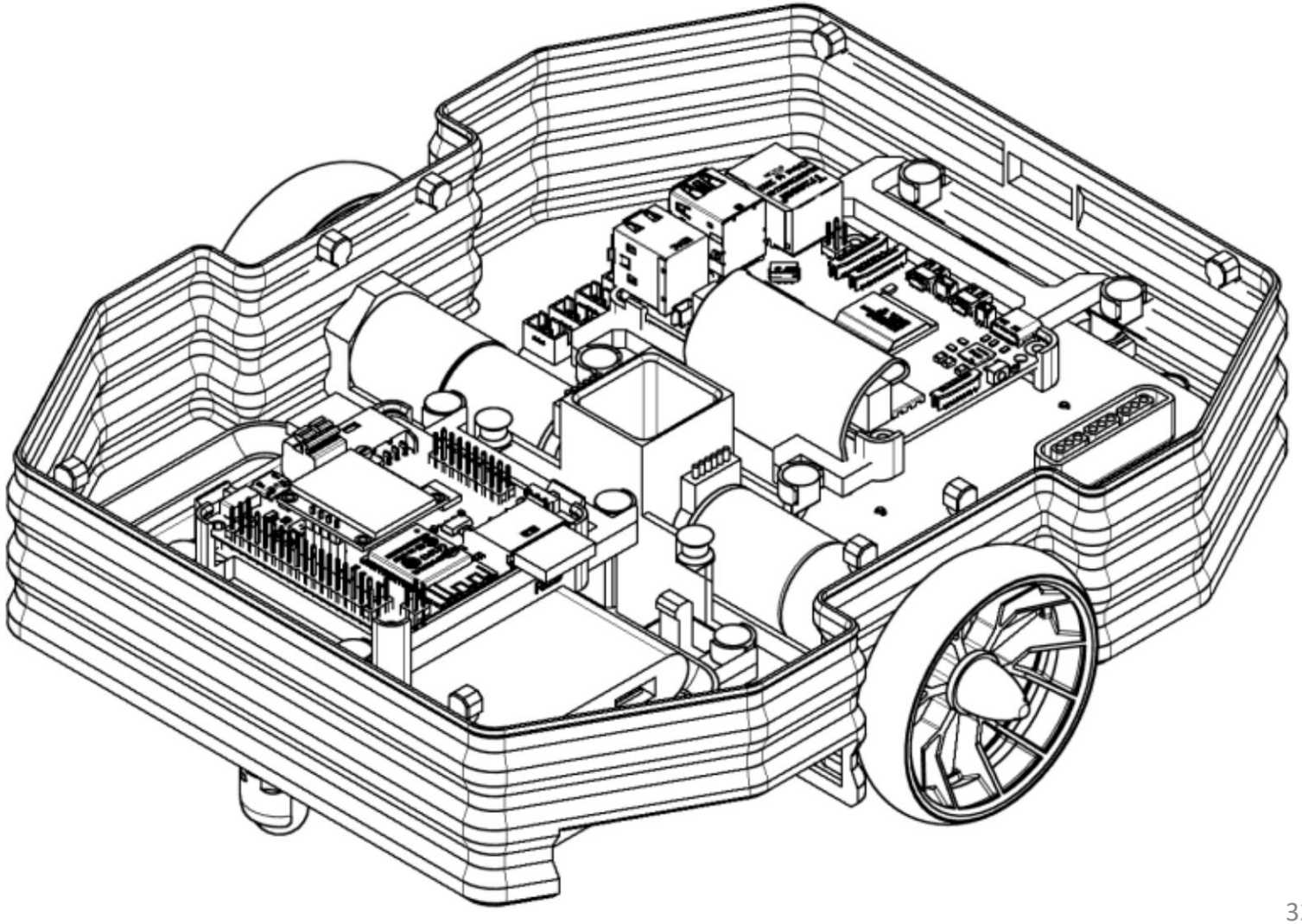
1. Download the Raspberry Pi Imager form the official website.
2. Open the Raspberry Pi Imager and insert the SD into your PC.
3. Choose Raspberry Pi 5 in destination device.
4. To choose the OS click on: Other General Purpose OS -> Ubuntu -> Ubuntu Desktop 24.04 LTS (64-bit).
5. Choose the SD as the destination storage.
6. Click on next and flash the SD.
7. Configure Ubuntu 24.04 generalities such as language, keyboard, wifi connections etc.



RPi5 Electrical Connections

The figure shows the final electrical connections between the Raspberry Pi 5, the Hackerboard, Yahboom's arm driver and the battery.

FALTA LINK A GIT

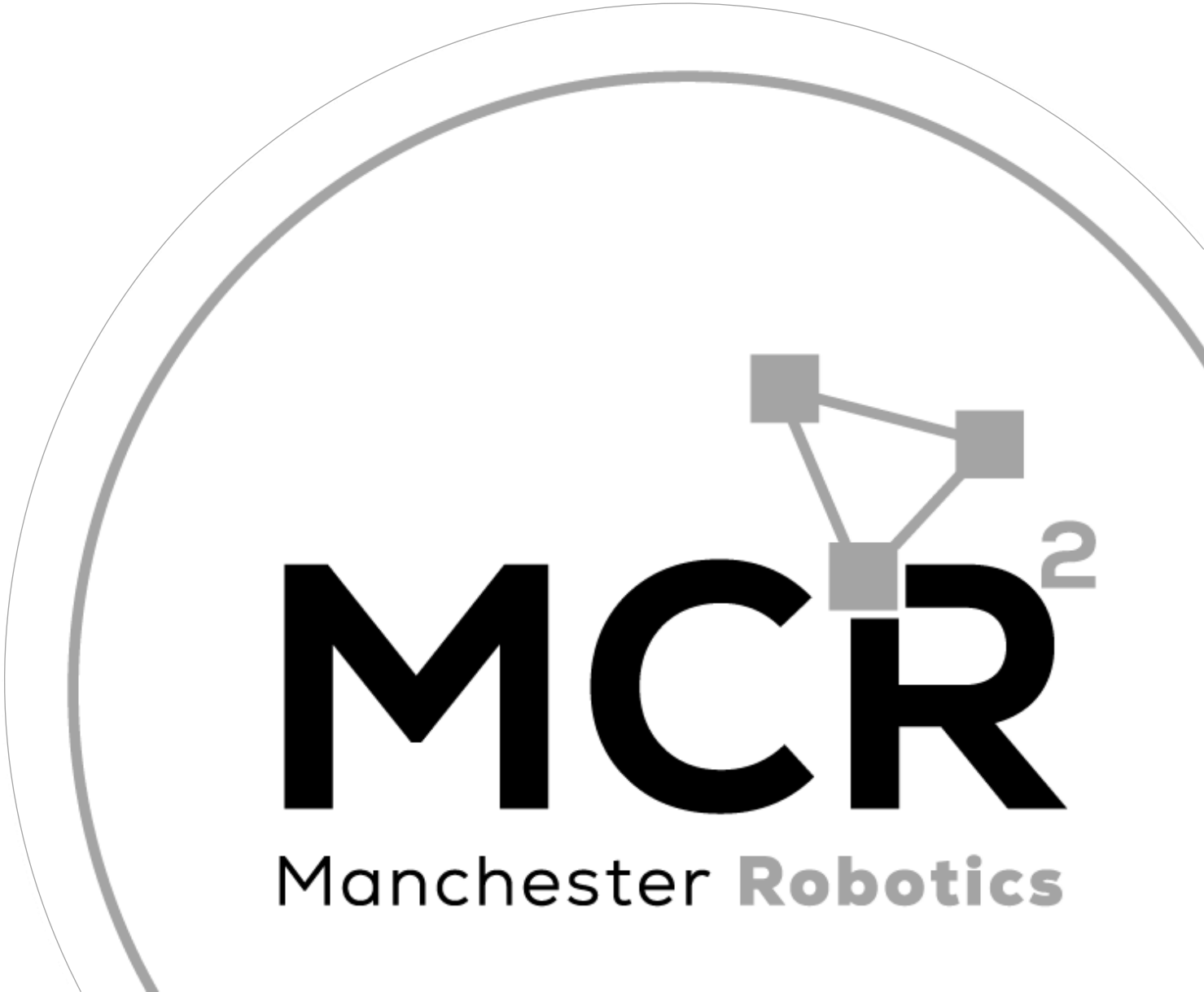


For details in connections and assembly please refer to the assembly manual.

NVIDIA Jetson Orin Nano®

Connections and configurations

{Learn, Create, Innovate};



NVIDIA Jetson Orin Nano

General characteristics

- **AI Performance**: Up to **67 INT8 TOPS** (Trillion Operations Per Second) with sparsity enabled.
- **Graphics (GPU)**: NVIDIA Ampere architecture with **1,024 CUDA Cores** and **32 Tensor Cores**.
- **Processor (CPU)**: 6-core Arm Cortex-A78AE v8.2 64-bit CPU (Clocked up to 1.7 GHz).

- **Memory (RAM)**: **8 GB 128-bit LPDDR5** with a fast memory bandwidth of **102 GB/s**.
- **Display Output**: 1 × DisplayPort 1.2 connector *(Note: HDMI is not natively supported without a passive/active adapter)*.
- **USB Ports**: 4 × USB 3.2 Gen 2 Type-A ports (10Gbps) and 1 × USB Type-C port (for data/recovery mode only).
- **Camera Interface**: 2 × MIPI CSI-2 (22-pin) camera connectors supporting up to 4 cameras.

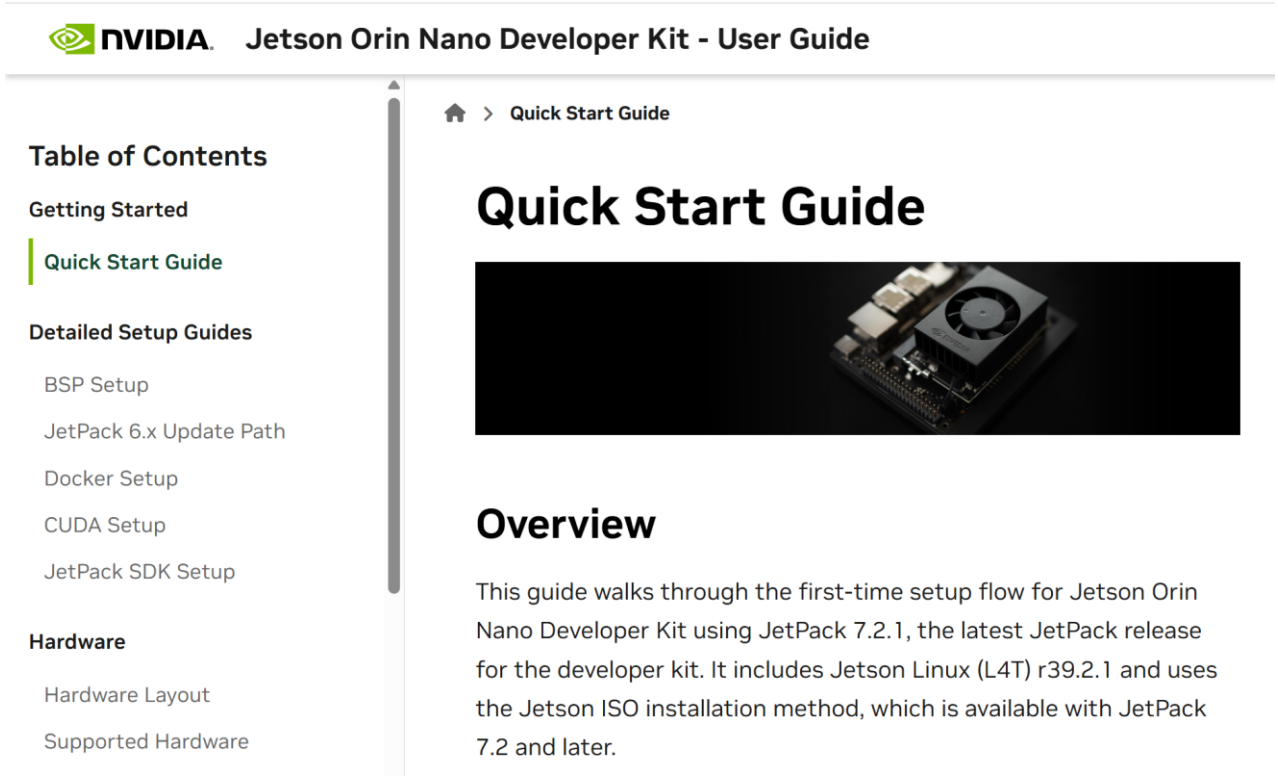
- **Wi-Fi / Bluetooth**: Included via the M.2 Key E wireless module card.
- **Ethernet**: 1 × Gigabit Ethernet (10/100/1000 Mbps) port.



Orin Nano OS Installation

Ubuntu 24.04 install instructions

1. Follow the official [Jetson Orin Nano Developer Kit – User Guide] to set up the operating system.
    1. The official webpage explains how upgrade the Orin's JetPack SDK, it also explains how to create a bootable USB with an ISO to setup the computer.
    2. The bootable ISO mentioned in the Jetson Orin Nano User Guide is also available in our GitHub.
2. If your when installing the OS via the bootable USB you encounter a catastrophic problem at the end of the configurations (right before the reboot) follow the troubleshoot guide to finish configuring the OS manually.
3. Configure Ubuntu 24.04 generalities such as language, keyboard, wifi connections etc.



OS Install Troubleshoot

“An error occurred. Press enter to start a shell” message at the end of the ISO configuration.

This error happened since the predefined TNSPEC (board name and specs) is not properly set on NVIDIA’s ISO. Use the following steps to correct it.

1. Press enter to start a shell.
2. Mount the following discs

```
mount --bind /dev /target/dev
mount --bind /dev/pts /target/dev/pts
mount --bind /proc /target/proc
mount --bind /sys /target/sys
mount -t efivarfs efivarfs /target/sys/firmware/efi/efivars
```

3. Check the current value of the TNSPEC

```
cat /target/etc/nv_boot_control.conf
```

4. Correct the TNSPEC value.

```
sed -i 's/^TNSPEC .*/TNSPEC 3767-300-0005-R.1-1-1-jetson-orin-nano-devkit-/' /target/etc/nv_boot_control.conf
```

5. Enter chroot, reconfigure and rebuild the configuration package that failed.

```
chroot /target /bin/bash
dpkg --configure -a
```

6. Configure the user.

```
adduser puzzlearm
```

7. Add the user to superusers

```
usermod -aG sudo puzzlearm
groups puzzlearm
```

8. Exit, unmount discs, poweroff, remove the ISO.

```
exit
umount /target/sys/firmware/efi/efivars
umount /target/sys
umount /target/proc
umount /target/dev/pts
umount /target/dev
```

sources Troubleshoot

Ubuntu is not able to locate sources when running install commands.

When this error happens it is necessary to restore the sources original file and finish some configurations manually

1. Open a terminal.
2. Make a security copy of the sources configurations file.

```
sudo cp /etc/apt/sources.list.d/ubuntu.sources
/etc/apt/sources.list.d/ubuntu.sources.empty.bak
```

3. Restore the original file from the system's backup file.

```
sudo cp /etc/apt/sources.list.d/ubuntu.sources.curtin.orig
/etc/apt/sources.list.d/ubuntu.sources
```

4. Try to update packages

```
sudo apt update
```

5. If step 4 fails, install nano

```
sudo apt install nano
```

6. Open the sources file

```
sudo nano /etc/apt/sources.list
```

7. Comment the following line on the sources file using a #. Then save the file with ctr + O, enter and ctr + X

```
deb [check-date=no] file:///tmp/cdrom/ noble main
restricted
```

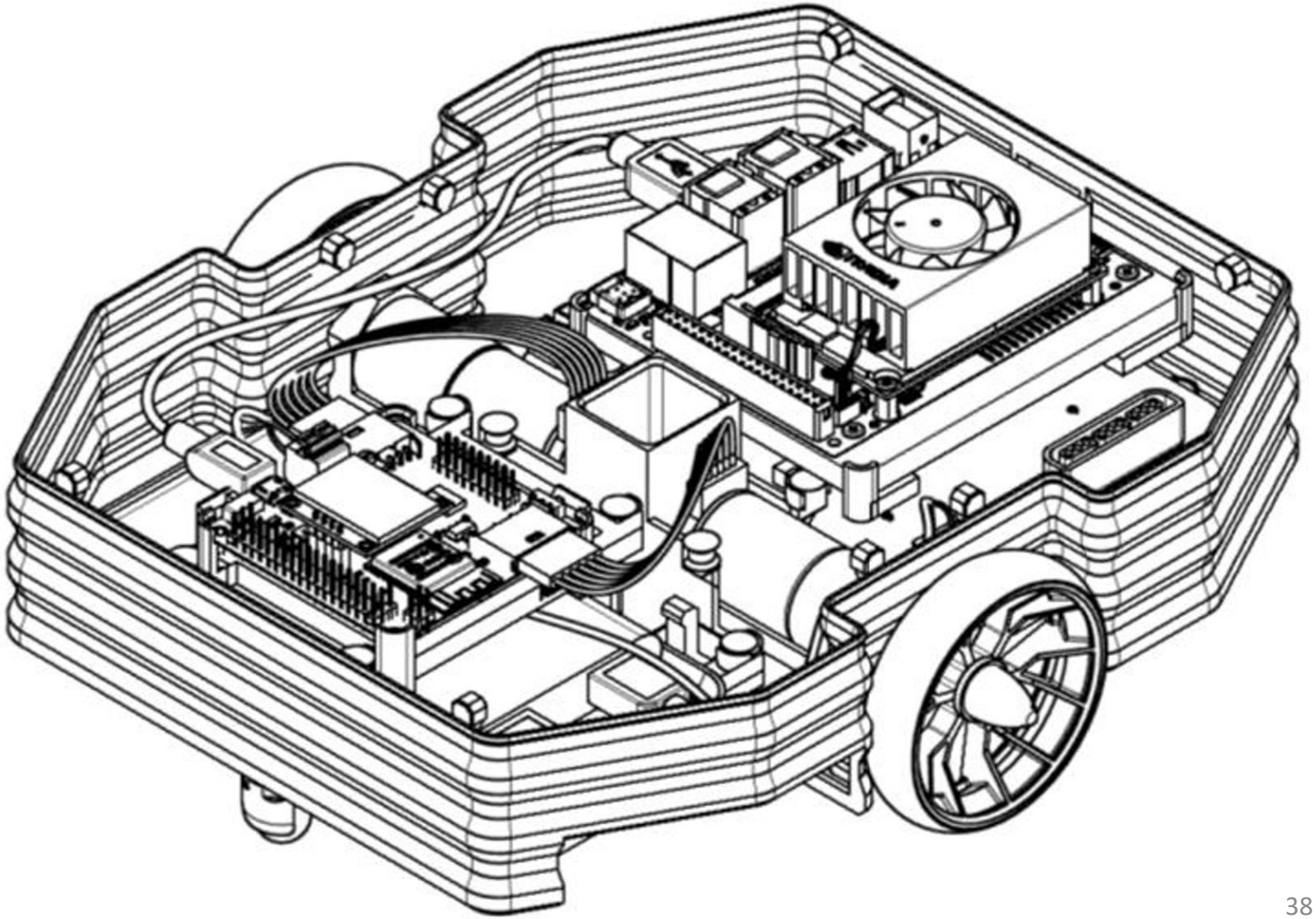
8. Update packages

```
sudo apt update
```

Orin Electrical Connections

The figure shows the final electrical connections between the NVIDIA Jetson Orin Nano, the Hackerboard, Yahboom's arm driver and the battery.

FALTA LINK A GIT

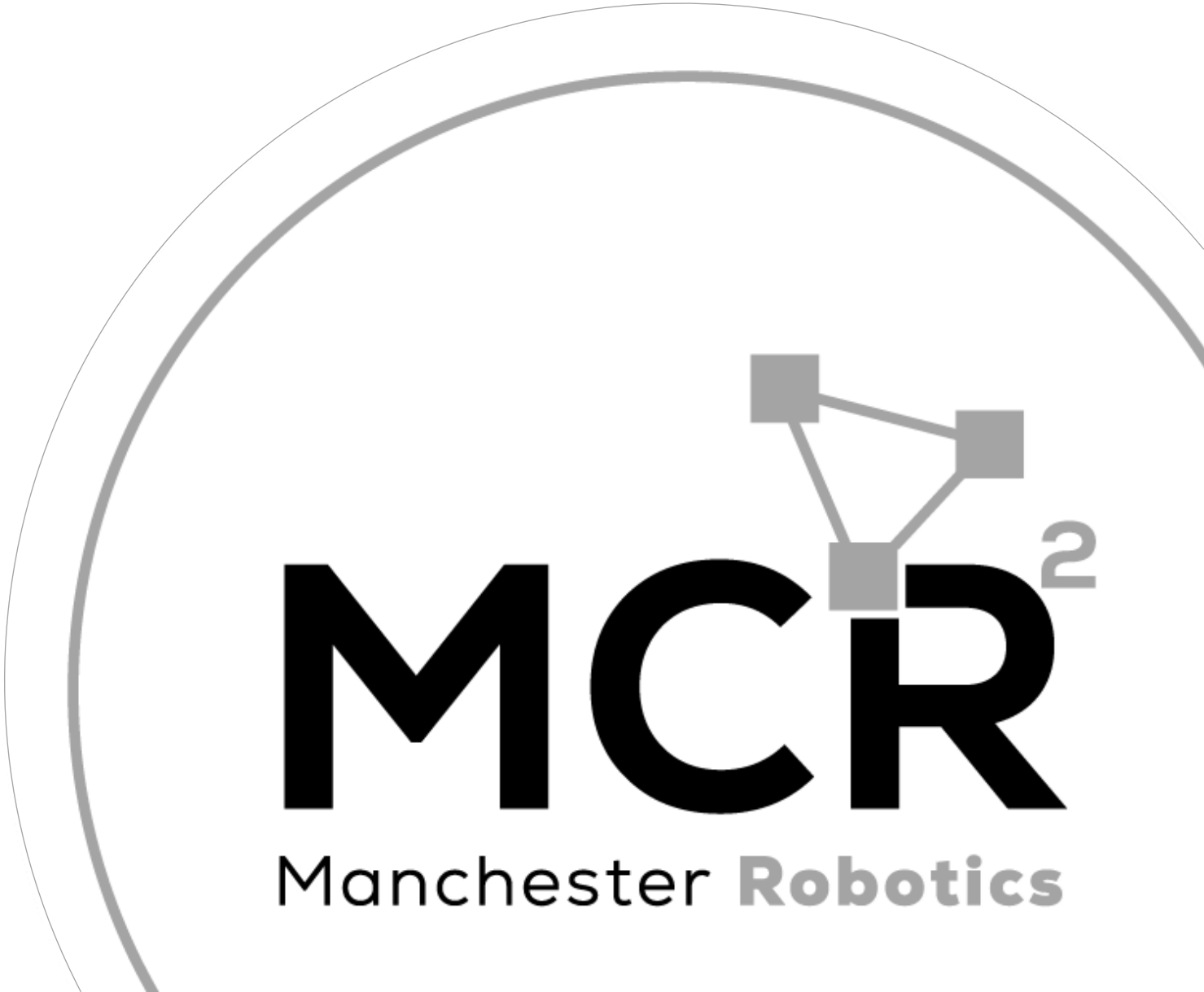


For details in connections and assembly please refer to the assembly manual.

General Configuration

Connections and configurations

{Learn, Create, Innovate};



ROS2 Jazzy and dependencies installation

ROS2 Jazzy Installation

1. Follow the official ROS2 Jazzy install instructions for Ubuntu to setup your ROS2 environment.

Install the following dependencies and packages

1. Python Pip
2. Numpy
3. Matplotlib
4. Pandas
5. Ultralytics
6. Tensorflow
7. Open CV
8. Open CV Contrib
9. CV Bridge
10. Net-tools
11. Smbus
12. Learning Joy



Yahboom Arm Libraries

Yahboom Arm Libraries Installation

1. Download the Yahboom ArmLib Python Module from MCR2 GitHub.
2. Extract all files in your computer and enter the 0.py_install directory
3. Install the python API to move the robotic arm with the following command in the terminal.

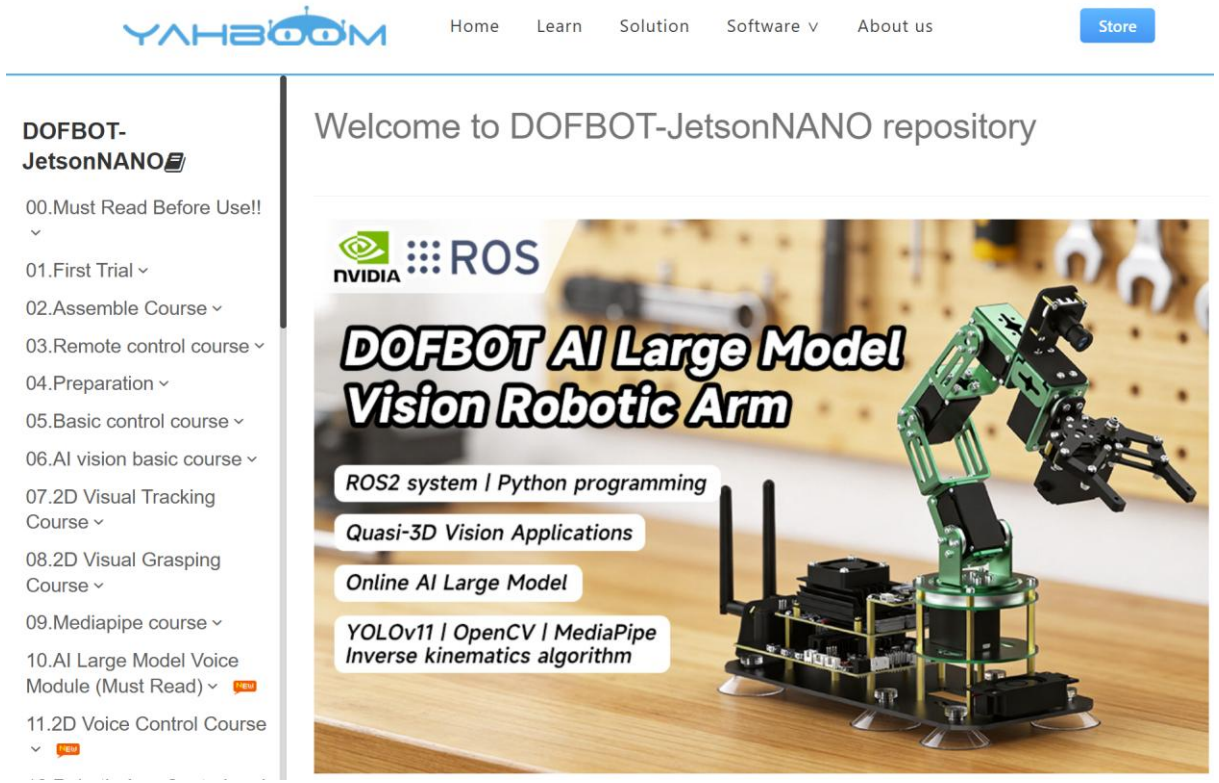
```
sudo python3 setup.py install
```

4. Verify the installation visualizing the installed modules.

```
pip3 list
```

Output:

```
Arm-Lib==0.0.4
```

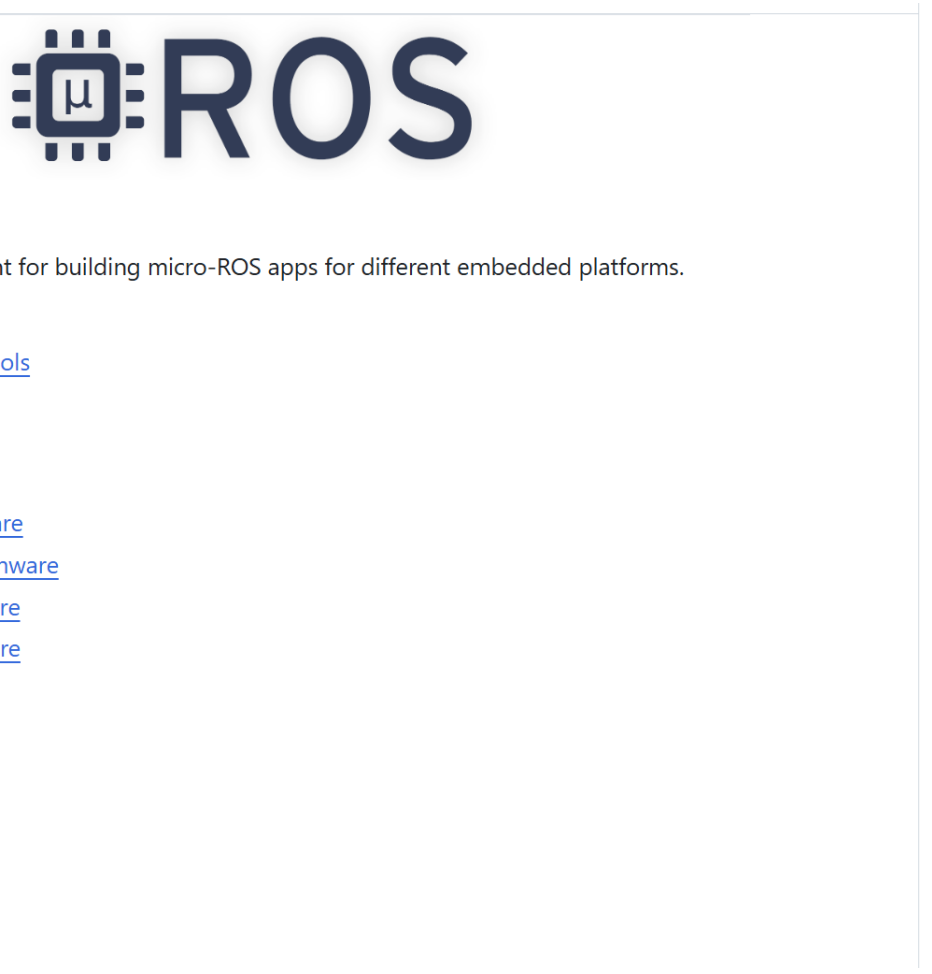


More information on the robotic arm here.

Micro ROS Libraries

Micro ROS Libraries Installation

1. Visit the official [Micro ROS repository in GitHub] to install and build the micro-ROS agent for ROS2 Jazzy.



This ROS 2 package is the entry point for building micro-ROS apps for different embedded platforms.

- Supported platforms
  - Standalone build system tools
- Dependencies
- Quick start
- Building
  - Creating micro-ROS firmware
  - Configuring micro-ROS firmware
  - Building micro-ROS firmware
  - Flashing micro-ROS firmware
- Building micro-ROS-Agent
- Contributing
- Purpose of the Project
- License
- Known Issues / Limitations
- Papers

MCR2 Packages

FALTAN LINK A GIT

Puzzlebot Package

1. Download the Puzzlebot package from MCR2 GitHub
2. Place it inside your current ros2 workspace and unzip the folder.
3. Compile the packages and source.

Puzzlearm Package

1. Download the Puzzlearm package from MCR2 GitHub
2. Place it inside your current ros2 workspace and unzip the folder.
3. Compile the packages and source.

Testing MCR2 Packages

Micro ROS Tests and Hackerboard Connection

- Test the ROS2 communication, run the following command (if not already running on the Jetson)

```
ros2 launch puzzlebot_ros micro_ros_agent.launch.py
```

- Echo the different topics using `ros2 topic echo`
  - Echo the topics `/VelocityEncR`, and `/VelocityEncR`, and rotate the wheels
  - The speed of the wheels should be displayed
- Publish to the command topics, the wheels should turn
  - If control mode 1 is used, publish to `/cmd_vel`
  - If control mode 2 is used, publish to `/VelocitySetR` and `/VelocitySetL`
  - If control mode 3 is used, publish to `/ControlL` and `/ControlL`
  - The control mode is changed on the Hackerboard webpage

Testing MCR2 Packages

Yahboom Robotic Arm Test

- Falta hacerlo paquete porque está como archivos solitos y falta el launch file

Raspberry Pi Camera

PREGUNTARLE A MARIO SI LO QUIERE O NO

- NVIDIA provides a package for interfacing with a CSI camera.
- To access the camera and test the connection type on a terminal

```
nvgstcapture-1.0
```

- To interface the camera with ROS, NVIDIA provides a ROS package that comes pre-installed on the Puzzlebot image
- For this package, several launch files are available. Only 2 are of interest to us:
  - ros_deep_learning video_viewer.ros2.launch
  - ros_deep_learning video_source.ros2.launch
- On your Jetson, run the command:

```
ros2 launch ros_deep_learning video_viewer.ros2.launch
```

- The camera view should be displayed on the screen.
- To only publish the image to a topic without any viewer, run the following command and ther run “ros2 topic list” command on another terminal.

```
ros2 launch ros_deep_learning video_source.ros2.launch
```

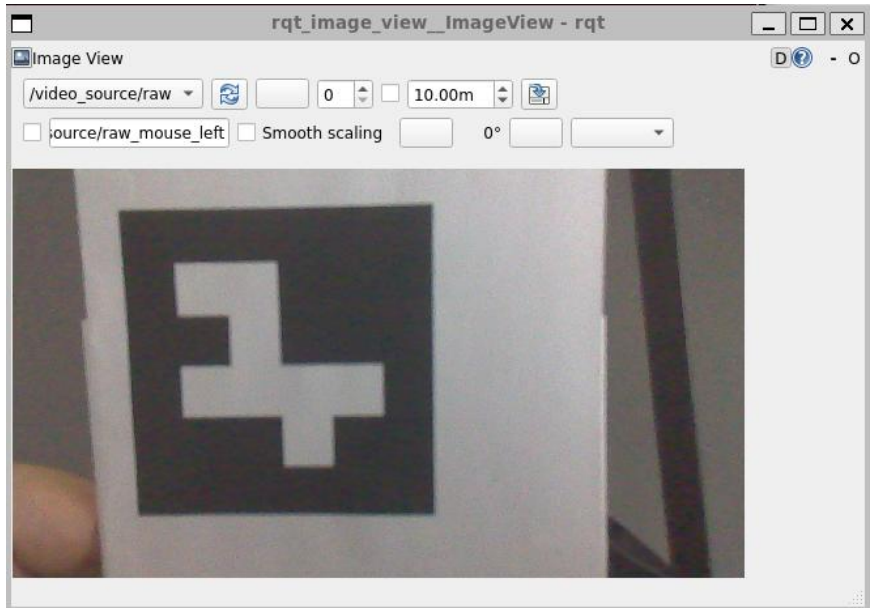
Raspberry Pi Camera

- MCR2 provides a launch file that contains the previous node and also publishes the camera information, required for different algorithms such as localisation.

```
$ ros2 launch puzzlebot_ros camera_jetson.launch.py
```

PREGUNTARLE A MARIO SI LO QUIERE O NO

```
mario@MarioPC:~$ ros2 topic list
/VelocityEncL
/VelocityEncR
/camera_info
/cmd_vel
/parameter_events
/robot_vel
/rosout
/video_source/raw
```

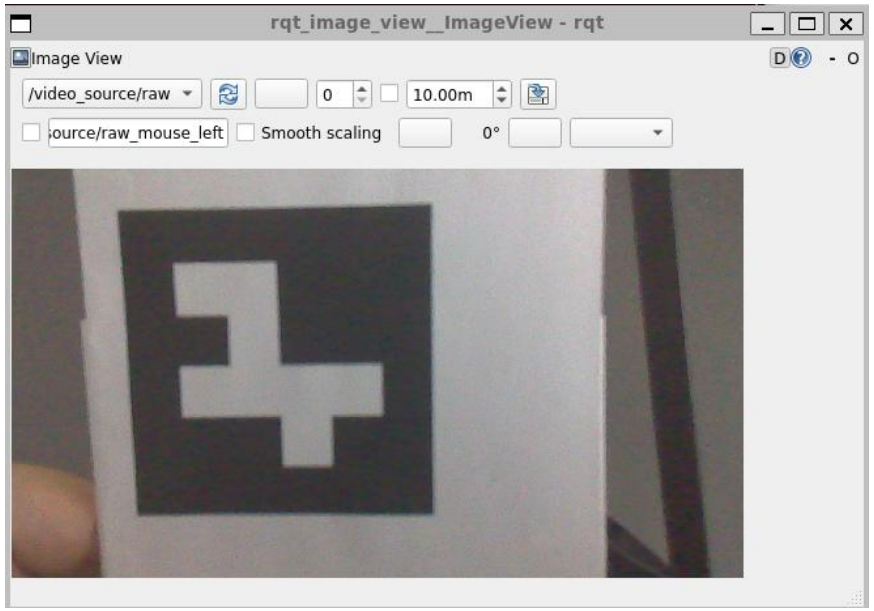


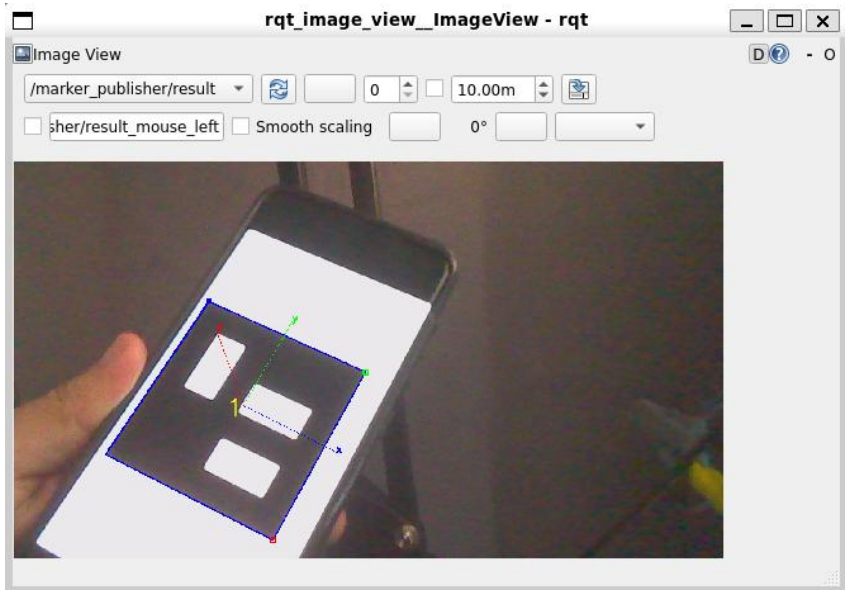
Raspberry Pi Camera

- MCR2 also provides April Tag marker detection

PREGUNTARLE A MARIO SI LO QUIERE O NO

```
$ ros2 launch puzzlebot_ros aruco_jetson.launch.py
```



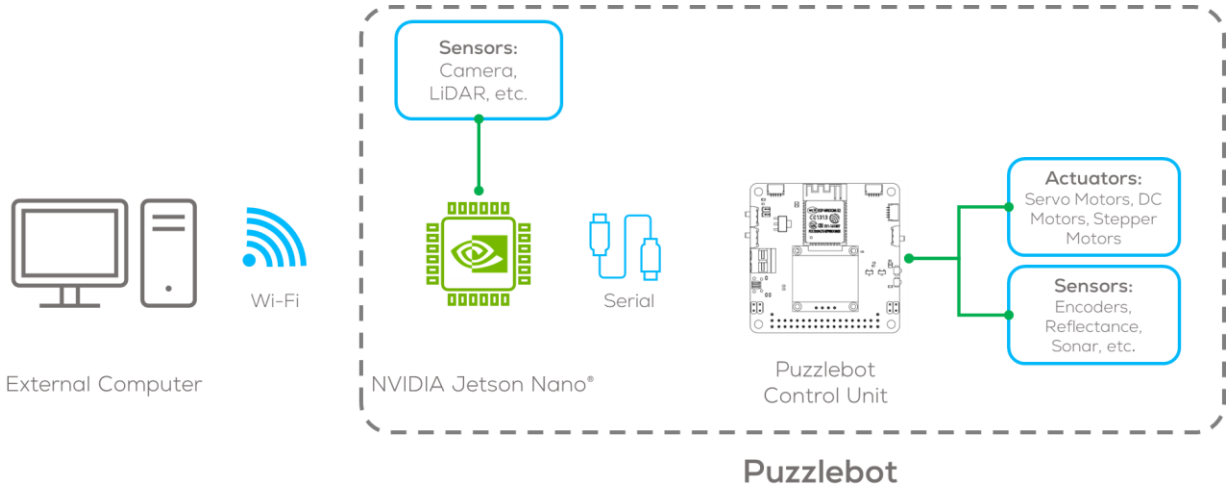



```
mario@MarioPC:~$ ros2 topic list
/camera_info
/marker_publisher/debug
/marker_publisher/markers
/marker_publisher/markers_list
/marker_publisher/result
/parameter_events
/rosout
/tf
/tf_static
/video_source/raw
```

Remote Access

1. It can be useful to control the Jetson from a remote PC, as the robot cannot be in motion and hooked up to a monitor
2. To do this, two ways of communication are available SSH or ROS Network. Both uses Wi-Fi to give your computer access to the Jetson.
3. For both connection in this tutorial, the Jetson needs to be configured as a Hotspot to generate its own access point.

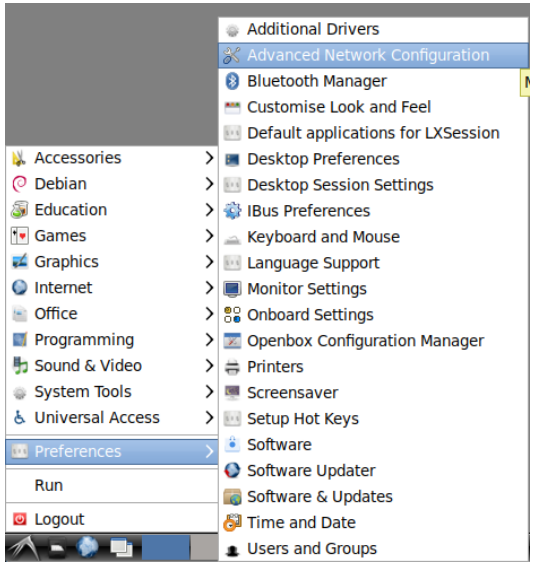
- **To prevent conflicts when many Jetsons are used, change the Network Name** to something unique (if applicable)

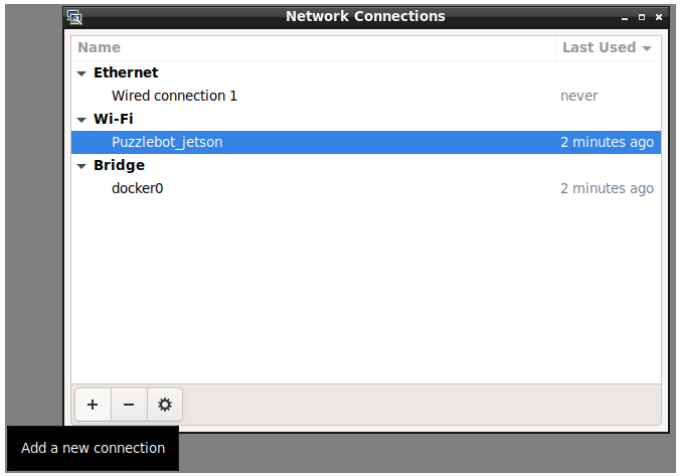


Hotspot Configuration

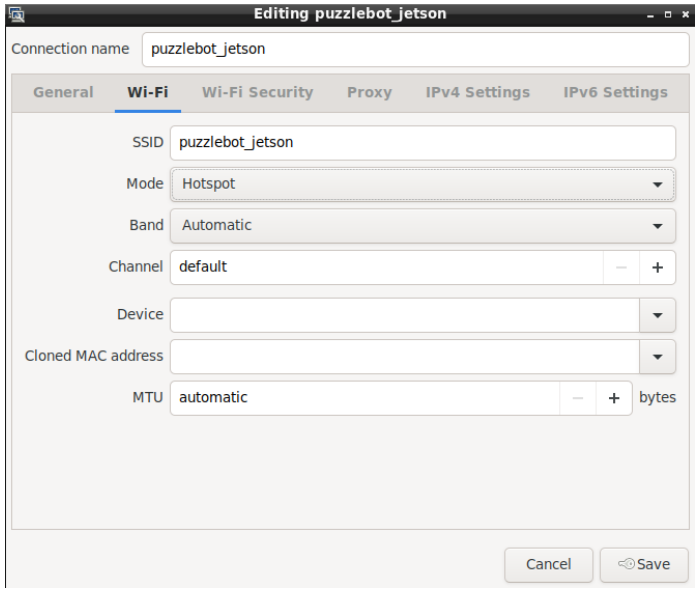
Hotspot Setup

1. Set up a hotspot by going to the Start Menu›› Preferences ›› Advanced Network Configuration
2. In the pop-up window click on the “+” signal to add a new connection.
3. Select on the pop-up window to Wi-Fi
4. Configure the name of the connection (you can use any name).
5. Configure the SSID (the name that you will see in other computers)
6. Change the mode to “Hotspot”









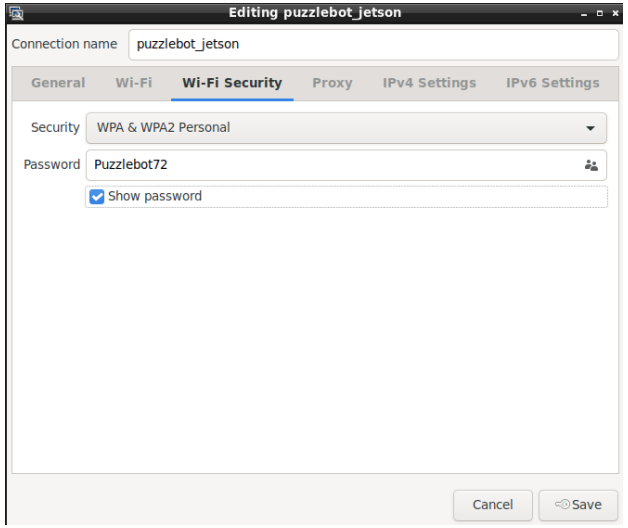
Hotspot Configuration

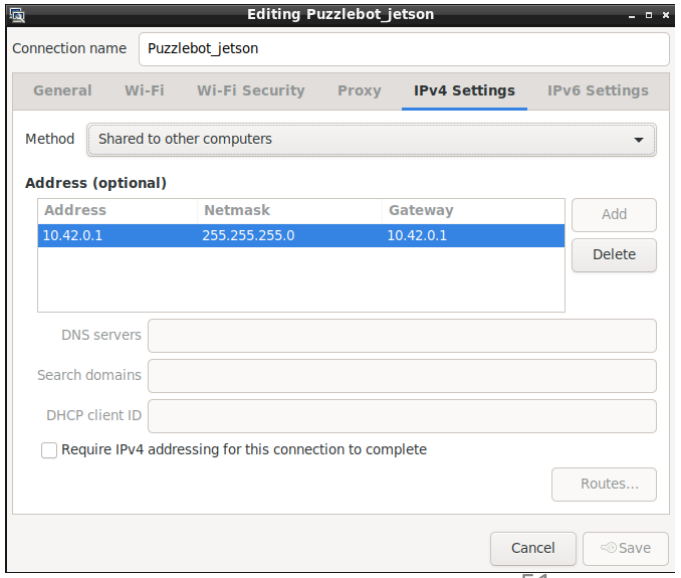
Hotspot Setup

7. In the “Wi-Fi” security tab establish a password for your own robot.

8. On the tab IPv4 setting configure an static IP address to connect to the robot.
    - In the example the addresses selected are the following
        - Address: 10.42.0.1
        - Netmask: 255.255.255.0
        - Gateway: 10.42.0.1

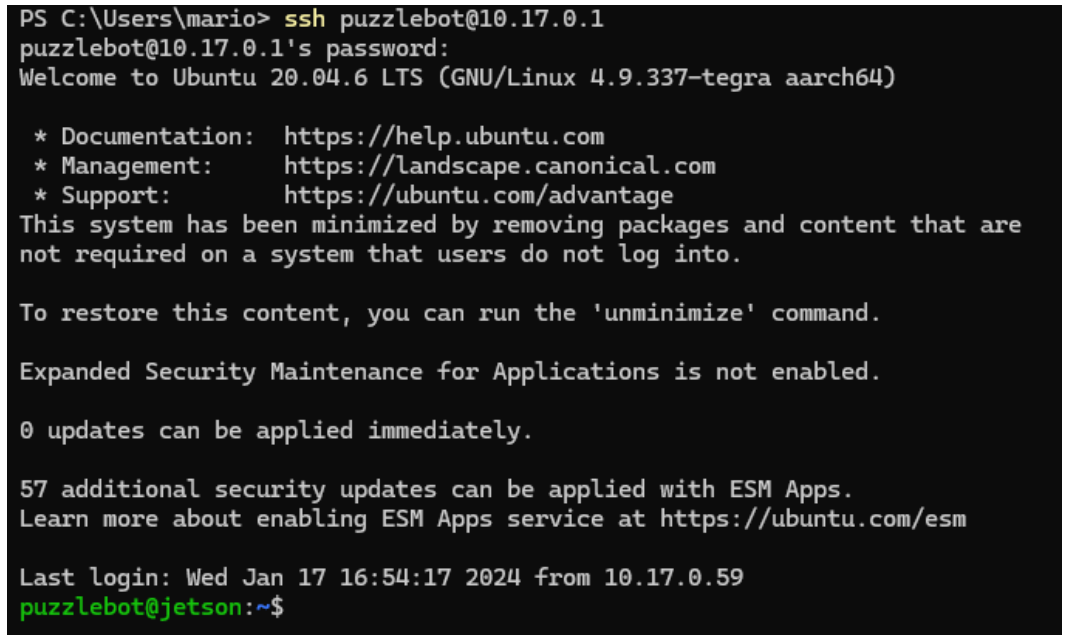
9. Save the configuration by clicking the “Save” button.





Remote Access

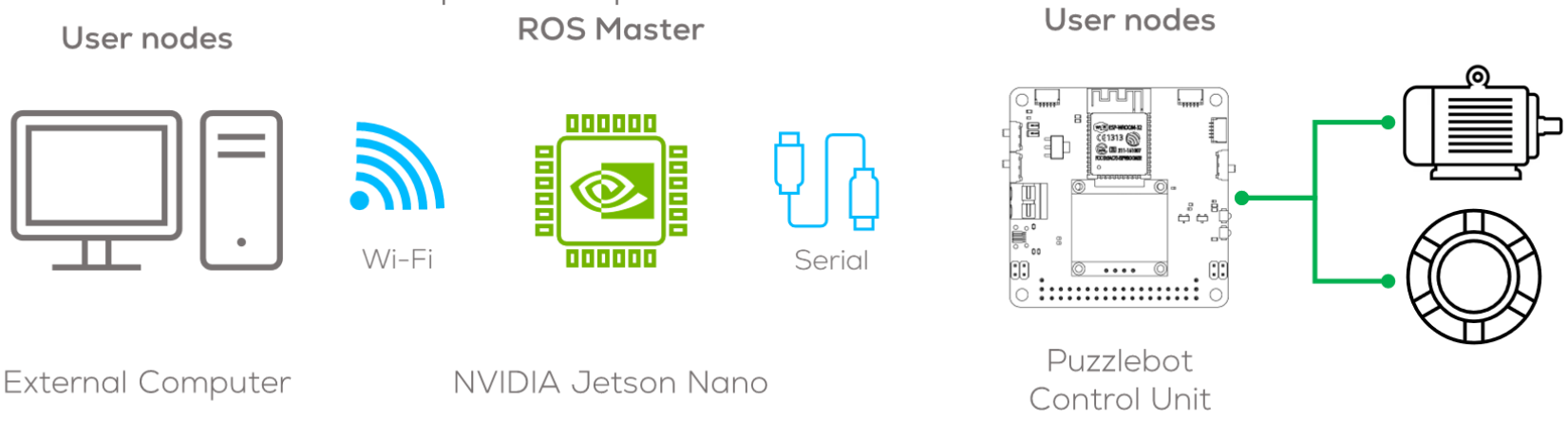
1. On the external PC (Windows or Ubuntu), open a Terminal window and type:
   - `ssh puzzlebot@10.42.0.1 (Configured IP Address)`
   - Password: `Puzzlebot72 (Configured Password)`
2. If prompted, type `yes` and then press enter.
3. This command window is now equivalent to one running on the Jetson.
4. Once any control code is written on the Jetson, it can be tested and debugged remotely via SSH, enabling the PuzzleBot to move around



Multi-device Communication with ROS (ROS network)

Introduction

- ROS facilitates communication between various devices, with ROS2 installed, on a shared network (No SSH).
- The Jetson Nano acts as an Access Point (AP) to provide this network.
- In the context of the Puzzlebot, by connecting to the Jetson's AP, other devices can communicate via ROS topics and publishers.



Multi-device Communication with ROS (ROS Network)

- From the host computer in Ubuntu with ROS2 Installed, connect to the “Hotspot” of the Jetson.
  - Make sure the IP address is correct using the command “ifconfig” on the terminal.
- ROS Should be able to detect the nodes, topics and subscribers.

- This connection is **not** a remote access into the Jetson like SSH, we cannot start nodes or run other commands
- However, it can be more useful, as we cannot easily display visualisations via SSH.

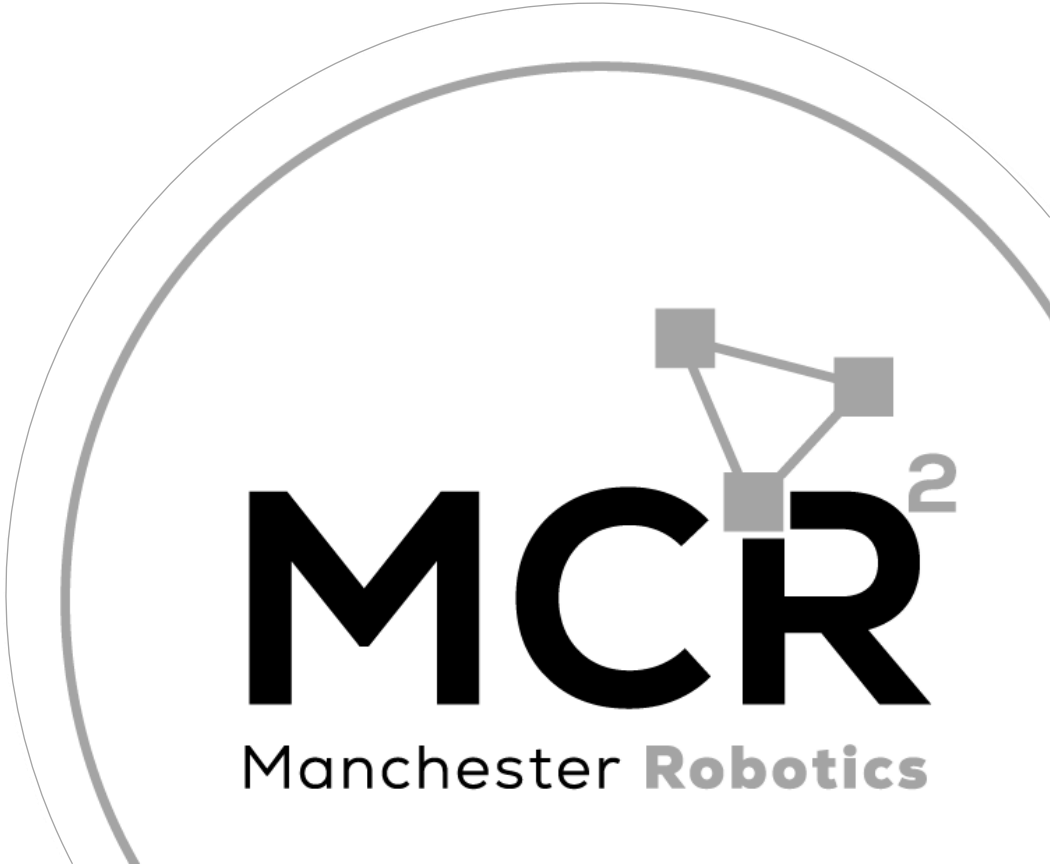
```
mario@MarioPC:~$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 10.17.0.31  netmask 255.255.255.0  broadcast 10.17.0.255
        ether 5e:bb:f6:9e:ee:fa  txqueuelen 1000  (Ethernet)
        RX packets 546  bytes 77617 (77.6 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 441  bytes 99820 (99.8 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 218  bytes 55509 (55.5 KB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 218  bytes 55509 (55.5 KB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

mario@MarioPC:~$ ros2 topic list
/VelocityEncL
/VelocityEncR
/cmd_vel
/parameter_events
/robot_vel
/rosout
```

Thank you

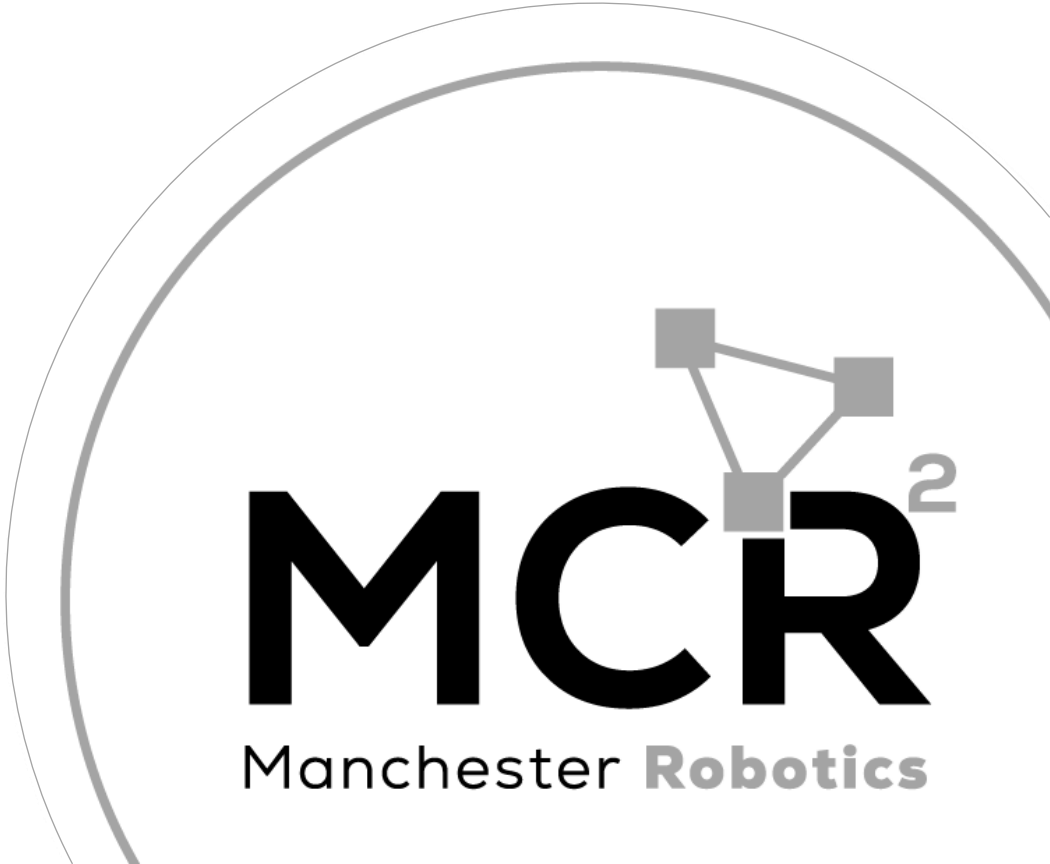
{Learn, Create, Innovate};



T&C

Terms and conditions

{Learn, Create, Innovate};



Terms and conditions

- *THE PIECES, IMAGES, VIDEOS, DOCUMENTATION, ETC. SHOWN HERE ARE FOR INFORMATIVE PURPOSES ONLY. THE DESIGN IS PROPRIETARY AND CONFIDENTIAL TO MANCHESTER ROBOTICS LTD. (MCR2). THE INFORMATION, CODE, SIMULATORS, DRAWINGS, VIDEOS PRESENTATIONS ETC. CONTAINED IN THIS PRESENTATION IS THE SOLE PROPERTY OF MANCHESTER ROBOTICS LTD. ANY REPRODUCTION, RESELL, REDISTRIBUTION OR USAGE IN PART OR AS A WHOLE WITHOUT THE WRITTEN PERMISSION OF MANCHESTER ROBOTICS LTD. IS STRICTLY PROHIBITED.*
- *THIS PRESENTATION MAY CONTAIN LINKS TO OTHER WEBSITES OR CONTENT BELONGING TO OR ORIGINATING FROM THIRD PARTIES OR LINKS TO WEBSITES AND FEATURES IN BANNERS OR OTHER ADVERTISING. SUCH EXTERNAL LINKS ARE NOT INVESTIGATED, MONITORED, OR CHECKED FOR ACCURACY, ADEQUACY, VALIDITY, RELIABILITY, AVAILABILITY OR COMPLETENESS BY US.*
- *WE DO NOT WARRANT, ENDORSE, GUARANTEE, OR ASSUME RESPONSIBILITY FOR THE ACCURACY OR RELIABILITY OF ANY INFORMATION OFFERED BY THIRD-PARTY WEBSITES LINKED THROUGH THE SITE OR ANY WEBSITE OR FEATURE LINKED IN ANY BANNER OR OTHER ADVERTISING.*